

Smart Thermal System for Patient Safety Monitoring: Full-Corpus Training, ONNX Export, and Quantization Analysis

Guy Chen · Yaniv Blau · Roy Lieberman

*Afeka Academic College of Engineering, Tel-Aviv — in collaboration with the Center for Mental
Health, Be'er Sheva*

Abstract

This report documents the full-corpus retraining of every detector in the Smart Thermal System pipeline (fire, human, and multi-view contact detection), together with ONNX export and fp16/bf16/int8 quantization benchmarks. Results below cover 8 detector(s) with at least one evaluated variant at generation time; missing variants are noted per section rather than silently omitted. Every metric is computed against the identical held-out waveshare_work test split for every detector and every variant, so numbers are directly comparable across the whole report. Mathematical derivations for each algorithm -- including the SVM KKT-eligibility argument -- are given in full, cross-checked against the shipped implementation rather than a textbook description of the method.

1. Dataset Composition

Dataset Composition

Two active data sources feed the full-corpus training and evaluation run described in this report: `waveshare_work` (real hardware, manually annotated) and `synth_room_1` through `synth_room_5` (synthetic, from a colleague-provided simulator). A third, real-hardware source was recorded but is not part of this dataset — its two recording dates each had independent data-quality problems (partial HDF5 corruption on one date, badly frozen sensor pixels on the other) discovered during verification, so it was excluded rather than used with a workaround.

	<code>waveshare_work</code>	<code>synth_room_1 ... synth_room_5</code>
Origin	Manually annotated (project team)	Synthetic simulator
Labels	Per-frame YOLO boxes (fire = class 0, person = class 1) + per-frame contact labels	Room-level event intervals
Resolution	80×62 (Waveshare _26984 sensor profile)	80×62, matches Waveshare_26984
Sessions	17 scenario folders	5 rooms × 1–3 recording dates each
Bounding boxes	Complete — all 17 scenarios	None — only room-level presence intervals

Because the synthetic sources carry no bounding boxes, they can supply frame-level presence labels (fire alert, human presence, contact event) for training/evaluating presence-based detectors, but never bounding-box regression or IoU-based evaluation — those are scoped to `waveshare_work`, the only source with real box ground truth.

Class balance

Across `waveshare_work`'s 17 scenarios (8,259 frame-slots):

Task	Positive	Negative	Ratio
Human	7,421 (89.9%)	838 (10.1%)	8.9 : 1
Fire	1,373 (16.6%)	6,886 (83.4%)	1 : 5.0
Contact	188 (6.8%)	2,565 (93.2%)	1 : 13.6

Across the full corpus (`waveshare_work`'s training scenes plus every synthetic session), the contact task is far more imbalanced still: roughly 2.4% of examples are contact-positive versus 97.6% negative. This imbalance is directly relevant to the contact-detector results in this report (see the MV-STGCN discussion) and is compensated for during training via an inverse- frequency class weight, though as discussed below that compensation does not fully overcome the imbalance for every model.

Session-level train/test split

Held-out evaluation splits are by whole recording session (never by individual frame), so that near-duplicate consecutive frames from the same session cannot leak between the training and test sets. Each of the three detection tasks — fire, human, and contact — has its **own independent** split, not one split shared across all three, because a scene that is representative for one task is not necessarily representative for another (in particular, only 4 of `waveshare_work`'s 17 scenes contain any contact-positive frames at all, so the contact split is curated to guarantee its held-out set contains positive examples rather than risk a random draw with none):

Task	Train scenes	Held-out test scenes
Fire	14	1person cig, 1ppl_single_forever, man_light_cig

Human	14	2ppl_hug, 3pplhedroncolider, man_light_cig_1
Contact	14	2ppl_fight, 2ppl_hug, 3pplhedroncolider

Every detector that addresses a given task is evaluated against that task's identical held-out scenes — e.g. both HOG-SVM and MobileNet-SSD (human detection) are scored on the exact same 3 scenes, and both MV-STGCN and Thermo-X3D (contact detection) are scored on the exact same 3 scenes — so results are directly comparable across models within a task. The synthetic sessions (synth_room_1–5) contribute no held-out split of their own: per the training design, synthetic data is used wholesale for training (it has no bounding boxes to evaluate against in the first place), and all held-out evaluation is anchored to `waveshare_work`'s real, human-annotated ground truth.

Raw sensor encoding

Thermal frames are stored as 16-bit unsigned integers in centi-Celsius (`temp_C = raw / 100`), verified against known-plausible room-temperature readings and the simulator's own configured clipping range.

Mathematical Derivations of the Core Detection Algorithms

Smart Thermal System for Patient Safety Monitoring — Beer Sheva Mental Health Center

This section derives, from first principles, the seven detection algorithms implemented in `thermal_algorithms/`. Each subsection matches the code actually shipped in the repository — file paths and parameter names are cited so the mathematics can be checked line-by-line against the implementation, rather than against a generic textbook description of the method. Where an existing evaluation report (`reports/*.tex`) already worked out part of a derivation, that content is adapted and cited rather than re-derived from scratch.

Reading index (by task)

Sections are numbered in the order they were written, not by task. By task they read:

Task	Sections
Human detection	§3 HOG-SVM, §4 MobileNet-SSD, §9 Adaptive Threshold
Fire detection	§1 Otsu thresholding, §2 Fire-SVM
Contact detection	§5 Geometric fusion, §6 MV-STGCN, §7 Thermo-X3D, §8 RBTCT

Every detector evaluated in this report has a section here, and every section here corresponds to a detector in the codebase. Two asymmetries are deliberate and are explained where they occur: §5 (Geometric) is derivation-only — it is not evaluated, because its homography could not be calibrated at this sensor's resolution — and §9 is numbered out of task order to avoid breaking existing cross-references to §8.

1. Otsu Thresholding — `fire_detection/otsu_utils.py::otsu_segment`

1.1 Formulation

Otsu's method converts a grayscale (here: temperature) image into a binary foreground/background mask by choosing the single intensity threshold k^* that best separates the pixel population into two classes, using only the image's own intensity histogram — no external calibration, no labeled data.

Given a float32 thermal frame `data` with per-frame minimum $d_{\min}$ and maximum $d_{\max}$, the implementation first rescales every pixel into $n_{\text{bins}} = 256$ integer bin indices:

$$\text{norm}(x) = \frac{x - d_{\min}}{d_{\max} - d_{\min} + \varepsilon} (n_{\text{bins}} - 1), \quad \varepsilon = 10^{-6}$$

and builds a histogram $h_0, h_1, \dots, h_{n_{\text{bins}}-1}$ of bin counts over the frame. Normalizing by the total pixel count $N = \sum_k h_k$ gives the empirical probability mass function

$$p_k = \frac{h_k}{N}, \quad k = 0, \dots, n_{\text{bins}} - 1, \quad \sum_k p_k = 1.$$

1.2 Derivation: between-class variance $\sigma_B^2(k)$

For a candidate threshold k , every pixel is assigned to class $C_0 = \{0, \dots, k\}$ ("background") or $C_1 = \{k+1, \dots, n_{\text{bins}} - 1\}$ ("foreground"). Define the class occupation probabilities

$$\omega_0(k) = \sum_{i=0}^k p_i, \quad \omega_1(k) = \sum_{i=k+1}^{n_{\text{bins}}-1} p_i = 1 - \omega_0(k),$$

and the class means

$$\mu_0(k) = \frac{1}{\omega_0(k)} \sum_{i=0}^k i p_i, \quad \mu_1(k) = \frac{1}{\omega_1(k)} \sum_{i=k+1}^{n_{\text{bins}}-1} i p_i.$$

The code computes these via running cumulative sums (`w0 = cumsum(p)`, `mu_k = cumsum(idx * p)`), which is the numerically efficient form:

$$\omega_0(k) = \sum_{i \leq k} p_i, \quad \sum_{i \leq k} i p_i = \omega_0(k) \mu_0(k) \quad (\text{the code's } \text{mu_k}).$$

The total (frame-wide) mean is $\mu_T = \sum_i i p_i = \text{mu_k}[-1]$, and it decomposes as a weighted average of the class means for *any* threshold k :

$$\mu_T = \omega_0(k) \mu_0(k) + \omega_1(k) \mu_1(k).$$

Total variance decomposition. The total intensity variance of the histogram can always be split into a *within-class* term and a *between-class* term:

$$\sigma_T^2 = \sigma_W^2(k) + \sigma_B^2(k).$$

with the within-class and between-class terms

$$\sigma_W^2(k) = \omega_0 \sigma_0^2 + \omega_1 \sigma_1^2,$$

$$\sigma_B^2(k) = \omega_0 (\mu_0 - \mu_T)^2 + \omega_1 (\mu_1 - \mu_T)^2.$$

Since σ_T^2 does not depend on k (it is a property of the whole histogram, fixed once the frame is fixed), *maximizing* $\sigma_B^2(k)$ over k is exactly equivalent to *minimizing* $\sigma_W^2(k)$ — Otsu's criterion simultaneously makes the two classes as internally homogeneous as possible *and* as mutually separated as

possible, because those are the same objective viewed from opposite sides of Eq. (2).

Closed form used in the code. Substituting $\mu_1 = (\mu_T - \omega_0 \mu_0) / \omega_1$ from Eq. (1) into the between-class term of Eq. (2) and simplifying (a standard algebraic reduction — see e.g. Otsu, 1979) collapses the two-term expression into a single ratio that needs only $\omega_0(k)$ and the cumulative first moment:

$$\sigma_B^2(k) = \omega_0(k) \omega_1(k) (\mu_1(k) - \mu_0(k))^2 = \frac{(\mu_T \omega_0(k) - \text{mu_k}(k))^2}{\omega_0(k) \omega_1(k)}.$$

This is precisely the `sigma_b2` array computed in the code:

```
sigma_b2 = (mu_t * w0 - mu_k) ** 2 / (w0 * w1)
# guarded where w0>0 and w1>0
```

Equation (3) is a function of k alone (all other quantities are frame-wide constants), so it can be evaluated for every one of the 256 candidate thresholds in $O(n_{\text{bins}})$ time given the cumulative sums — no per-threshold pass over the pixels is needed. The optimal threshold is

$$k^* = \arg \max_k \sigma_B^2(k), \quad \text{threshold} = d_{\min} + \frac{k^*}{n_{\text{bins}} - 1} (d_{\max} - d_{\min}),$$

exactly `k_star = argmax(sigma_b2)` followed by the de-normalization back to physical temperature units. The binary mask is then `data >= threshold`.

1.3 Why this method is theoretically justified here

Otsu's threshold is the **maximum-likelihood boundary under a two-Gaussian mixture model with equal variances and equal priors** projected onto the 1-D intensity axis — i.e., it is the optimal separator when the frame genuinely contains two thermally distinct populations (e.g., "warm scene" vs. "hot blob"). It requires no labeled data and adapts automatically to per-frame dynamic range, which matters here because ambient ward temperature drifts and different fire/ignition scenes have very different absolute temperature scales. This makes it a reasonable **region proposal** stage (as used by `FireSVMDetector`, §2) even though — as `reports/fire_detection_report.tex` documents — a global two-class assumption is fragile when the "hot" class is dominated by warm human bodies rather than by the (much smaller, much hotter) fire blob; that failure mode motivated the hot-pixel bypass fix described there, distinct from the pure derivation given here.

1.4 Morphological post-processing

After thresholding, the code applies

```
eroded = cv2.erode(raw_mask, morph_kernel,
iterations=1)
dilated = cv2.dilate(eroded, morph_kernel,
iterations=2)
```

Erosion ($A \ominus B = \{z : B_z \subseteq A\}$, i.e. keep a foreground pixel only if the **entire** structuring element B centered there is inside the mask) shrinks every foreground blob by one structuring-element radius. A single hot pixel with no hot neighbors — sensor noise, a stray dead-pixel glitch, a one-pixel thermal reflection — fails the "entire kernel inside" test and is deleted outright. This is a **noise-removal** step: it enforces spatial coherence by requiring a candidate region to have some minimum extent, not just isolated hot pixels.

Dilation ($A \oplus B = \{z : B_z \cap A \neq \emptyset\}$, i.e. grow a region wherever the kernel touches any foreground pixel) then grows the surviving blobs back out, applied **twice** so that the net effect on a surviving multi-pixel blob is a **one-kernel-radius net expansion** relative to the original mask (one erosion undone, plus one extra dilation). This **consolidates** nearby fragments of what is physically a single hot object — e.g. a blob

that Otsu's hard threshold happened to split into two disconnected pixel clusters due to noise — into one connected component, and slightly pads the region so `extract_blobs`'s bounding box does not clip the true extent of the hot object. Erosion-then-dilation in this order (rather than dilation-then-erosion, i.e. morphological **closing**) is deliberately asymmetric: it guarantees single/isolated-pixel noise is permanently removed (erosion is a strict, one-way filter here) while the surviving structure is **grown**, not just closed — appropriate for a detector whose failure mode of most concern (per §1.3) is a fire blob that is small to begin with.

2. FireSVMDetector — fire_detection/fire_svm.py

2.1 Formulation

`FireSVMDetector` is a two-class classifier (`SAFE = -1` vs. `ACTIVE_COMBUSTION = +1` in the classical SVM sign convention; the code uses sklearn's $\{0, 1\}$ label convention internally, which is equivalent) over a **6-dimensional hand-engineered feature vector** computed from the hottest connected component returned by `otsu_segment + extract_blobs` (§1):

$$x = (T_{\max}, T_{\text{mean}}, T_{\text{std}}, \text{area}, \text{skew}, \text{kurt}) \in \mathbb{R}^6.$$

Skewness and kurtosis are the standardized third and fourth central moments of the blob's pixel temperatures $\{t_i\}_{i=1}^n$ (biased/population form, matching `scipy.stats.skew/kurtosis` defaults):

$$\text{skew} = \frac{\frac{1}{n} \sum_i (t_i - \bar{t})^3}{(\frac{1}{n} \sum_i (t_i - \bar{t})^2)^{3/2}}, \quad \text{kurt} = \frac{\frac{1}{n} \sum_i (t_i - \bar{t})^4}{(\frac{1}{n} \sum_i (t_i - \bar{t})^2)^2} - 3.$$

These capture **shape** rather than magnitude: a flame's pixel-temperature distribution is asymmetric (positive skew — a hot core trailing to cooler edges) and heavy-tailed/peaked (positive excess kurtosis — flicker produces outlier-hot pixels against a cooler bulk), whereas a warm human torso at a fairly uniform $\sim 36^\circ\text{C}$ is close to symmetric and mesokurtic. This is why the model can distinguish "small hot flickering thing" from "large steady warm thing" even when the two happen to share a

similar $T_{\max}$ (adapted from `reports/checkpoint_report.tex`, "Skewness and kurtosis" paragraph, which documents the identical six-feature vector).

Before classification, features are z-scored using `StandardScaler` fit on the training set: $\hat{x}_j = (x_j - \mu_j)/\sigma_j$.

2.2 The primal soft-margin QP

`FireSVMDetector` trains an `sklearn.svm.SVC` — a soft-margin kernel SVM (support vector machine: a classifier that finds the hyperplane separating the two classes with the largest possible margin to the nearest training points, rather than just any separating hyperplane). For labeled training pairs (x_i, y_i) , $y_i \in \{-1, +1\}$, and a feature map $\phi(\cdot)$ (identity for the linear kernel, the implicit RBF map for `kernel="rbf"`, the code's default), the classifier seeks a hyperplane $w^\top \phi(x) + b = 0$ that separates the classes with maximum margin while tolerating some misclassification. Introducing one slack variable $\xi_i \geq 0$ per training point to allow margin violations, the **primal problem** is the convex quadratic program (QP — a *quadratic program* minimizes a quadratic objective subject to linear constraints; convex QPs like this one can be solved to global optimality efficiently, which is exactly what makes the derivation below possible)

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n.$$

$\xi_i = 0$ means point i is correctly classified and outside the margin corridor; $0 < \xi_i < 1$ means it is correctly classified but inside the corridor; $\xi_i > 1$ means it is misclassified. C (the constructor's `svm_C`, default 1.0) is the unit price of a margin violation: it is the single knob trading margin width against training-set fit, and is exactly the inverse of an L_2 weight-decay coefficient when Eq. (4) is rewritten in unconstrained hinge-loss form,

$$\min_{w, b} \sum_i \max(0, 1 - y_i(w^\top \phi(x_i) + b)) + \frac{1}{2C} \|w\|^2,$$

i.e. ****hinge loss plus ℓ_2 regularization****, with $C \rightarrow \infty$ recovering the hard-margin SVM and $C \rightarrow 0$ making all violations free (adapted from `reports/checkpoint_report.tex`, "Why C is a

regularizer").

2.3 The Lagrangian and the dual problem

Introduce Lagrange multipliers $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for the slack non-negativity constraints:

$$\mathcal{L}(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^\top \phi(x_i) + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

Setting the gradient of $\mathcal{L}$ with respect to each primal variable to zero (stationarity — see §2.4) gives

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_i \alpha_i y_i \phi(x_i) = 0 \Rightarrow w = \sum_i \alpha_i y_i \phi(x_i),$$

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_i \alpha_i y_i = 0 \Rightarrow \sum_i \alpha_i y_i = 0,$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \Rightarrow \mu_i = C - \alpha_i \Rightarrow 0 \leq \alpha_i \leq C.$$

Substituting (6)–(8) back into (5) eliminates w, b, ξ entirely (the hallmark of a QP dual) and leaves a maximization over α alone, in which the data enters *only* through pairwise inner products $\langle \phi(x_i), \phi(x_j) \rangle$:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle \phi(x_i), \phi(x_j) \rangle \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

This is the **dual problem**, adapted from `reports/checkpoint_report.tex` ("The dual and the kernel trick"), and is itself a convex QP (concave objective maximized, equivalently convex minimized, over a box-and-hyperplane feasible set) — solved by `sklearn's SVC` internally via **SMO** (Sequential Minimal Optimization), which repeatedly picks the pair (α_i, α_j) that most violates optimality and updates it to its closed-form constrained optimum. There is consequently no learning-rate, epoch count, or gradient-descent schedule for this model — training is an exact convex optimization, and the only capacity knobs are C and (for RBF) γ .

2.4 KKT conditions

For a general constrained optimization $\min_x f(x)$ s.t. $g_i(x) \leq 0$, $h_j(x) = 0$, the **Karush–Kuhn–Tucker (KKT) conditions** at a candidate optimum (x^*, λ^*, ν^*) are:

1. **Stationarity:**

$$\nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0.$$

2. **Primal feasibility:** $g_i(x^*) \leq 0$, $h_j(x^*) = 0$ for all i, j . 3. **Dual feasibility:** $\lambda_i^* \geq 0$ for all i . 4. **Complementary slackness:** $\lambda_i^* g_i(x^*) = 0$ for all i .

Mapping this template onto the soft-margin SVM of Eq. (4), with multipliers $\alpha_i \geq 0$, $\mu_i \geq 0$ for the two inequality constraint families respectively:

$$g_i^{(1)}(w, b, \xi) = 1 - \xi_i - y_i(w^\top \phi(x_i) + b) \leq 0, \quad g_i^{(2)}(\xi) = -\xi_i \leq 0.$$

- **Stationarity** — exactly Eqs. (6)–(8) above: $w = \sum_i \alpha_i y_i \phi(x_i)$, $\sum_i \alpha_i y_i = 0$, $\mu_i = C - \alpha_i$.
- **Primal feasibility** — $y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for every training point.
- **Dual feasibility** — $\alpha_i \geq 0$ and $\mu_i \geq 0$ (equivalently, via (8), $0 \leq \alpha_i \leq C$).
- **Complementary slackness** — two conditions, one per constraint family:

$$\alpha_i [y_i(w^\top \phi(x_i) + b) - 1 + \xi_i] = 0, \quad \mu_i \xi_i = 0.$$

Equation (10) is what gives the SVM its sparse **support-vector** structure and its three-way partition of the training set:

- If $\alpha_i = 0$: point i contributes nothing to w (Eq. 6) — it lies strictly outside the margin and is *not* a support vector.
- If $0 < \alpha_i < C$: then $\mu_i = C - \alpha_i > 0$, so by the second half of (10) $\xi_i = 0$; combined with the first half of (10), $y_i(w^\top \phi(x_i) + b) = 1$ exactly — the point sits **on** the margin boundary. This is what makes b recoverable in closed form from any such point.
- If $\alpha_i = C$: then $\mu_i = 0$, and ξ_i is free to be positive — the point lies inside the margin or is misclassified.

In the fitted SVC object, `_svm.n_support_/support_vectors_` are exactly the points with $\alpha_i > 0$; `reports/checkpoint_report.tex` reports 874 of 6,497 training points as support vectors ($\sim 13\%$) for this exact model, which is a direct empirical read-out of how many points satisfy $\alpha_i > 0$ under conditions (10).

2.5 Why we are eligible to use KKT here

KKT conditions are, in general, only **necessary** for optimality (for arbitrary smooth constrained problems), and are of little use if strong duality does not hold — a solution to the dual need not then equal the primal optimum, and the multipliers found need not certify anything about the primal solution. The reason the soft-margin SVM training procedure can **rely** on KKT — treat it as a two-way equivalence used both to derive the dual (§2.3) and to certify convergence during SMO — is a specific, checkable chain of facts about problem (4):

1. **The primal is a convex program.** The objective $(1/2)\|w\|^2 + C \sum_i \xi_i$ (Eq. 4) is a positive-semidefinite quadratic form in w plus a linear function of ξ — jointly convex in (w, b, ξ) . Every constraint, $y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$, is **affine (linear)** in (w, b, ξ) — hence both the feasible region is convex (an intersection of halfspaces) and the constraint functions are convex, satisfying the hypotheses required for KKT to be more than "necessary-only." 2. **Slater's condition holds trivially.** Slater's constraint qualification requires a point that is **strictly** feasible for every inequality constraint. Any (w, b) (e.g. $w = 0, b = 0$) combined with ξ_i chosen large enough — e.g. $\xi_i = 2$ for every i — satisfies $y_i(w^\top \phi(x_i) + b) = 0 \geq 1 - \xi_i = -1$ and $\xi_i = 2 > 0$ **strictly**, for **every** training set, with no assumption of linear separability. Soft-margin SVM training is feasible by construction (unlike the hard-margin SVM, whose feasible set can be empty for non-separable data) — this is precisely why the slack variables were introduced in the first place, and it is what guarantees Slater's condition holds for this problem regardless of the input data. 3. **Convexity + Slater ■ strong duality.** By convex-optimization theory (Slater's theorem), a convex problem satisfying Slater's condition has **zero duality gap**: the primal optimal value equals the dual optimal value, and — critically — a primal-dual pair $(w^*, b^*, \xi^*, \alpha^*, \mu^*)$ is optimal for both problems **if and only if** it satisfies the KKT conditions. This "if and only if" is exactly the upgrade from "KKT is necessary" to "**KKT is both necessary and sufficient**" that licenses the derivation in §2.3–2.4: we are not merely hoping a KKT point is the optimum, we have a theorem guaranteeing it is. 4. **Consequence for SMO.** Because KKT is

sufficient here, SMO's stopping rule — "no training example violates the KKT conditions by more than a numerical tolerance" — is a valid certificate of (near-)global optimality, not a heuristic. This is also why FireSVM training has no notion of local minima, random restarts, or optimizer instability: the feasible set is convex, the objective is convex, and any KKT point is *the* global optimum.

This is the complete eligibility argument requested: convex QP $\Rightarrow$ Slater trivially satisfied by a feasible soft-margin problem with $\xi \geq 0 \Rightarrow$ strong duality $\Rightarrow$ KKT necessary and sufficient.

2.6 The RBF kernel, Mercer's theorem, and the kernel trick

The dual (9) depends on the data only through inner products $\langle \phi(x_i), \phi(x_j) \rangle$. The **kernel trick** replaces this inner product with a kernel function $k(x_i, x_j)$ computed directly on the raw inputs, without ever constructing ϕ :

$$k(x, x') = \exp(-\gamma \|x - x'\|^2), \quad \gamma = \text{gamma} = \text{"scale"} \Rightarrow \gamma = \frac{1}{d \cdot \text{Var}(X)},$$

the RBF (radial basis function — a kernel that scores two points as similar exactly when they are geometrically close, with similarity decaying smoothly/exponentially as distance grows; also called the Gaussian kernel) kernel used by `FireSVMdetector(kernel="rbf")` (its default, though `kernel="linear"` is also exposed as a constructor option). This substitution is valid — i.e. the resulting optimization is still exactly the dual of *some* legitimate linear-in-feature-space SVM — precisely because of **Mercer's theorem**: any symmetric function $k(x, x')$ that is positive semi-definite (i.e. for any finite set of points $\{x_i\}$, the Gram matrix $K_{ij} = k(x_i, x_j)$ is PSD) can be written as $k(x, x') = \langle \phi(x), \phi(x') \rangle$ for *some* feature map ϕ into a (possibly infinite-dimensional) Hilbert space. The RBF kernel satisfies this PSD requirement (it is a Gaussian, whose Fourier transform — its spectral density — is non-negative everywhere, which is the Bochner-theorem condition for positive-definiteness). Consequently:

- We never need to compute or store $\phi(x)$ — every place $\langle \phi(x_i), \phi(x_j) \rangle$ appears in training (Eq. 9) or in the decision function (§2.7), it is replaced by $k(x_i, x_j)$, computed in $O(d)$ time on the original 6-D features.
- Taylor-expanding $e^{-\gamma \|x - x'\|^2}$ shows the implicit feature space contains monomials of *every* degree simultaneously — the RBF-SVM is a universal approximator over the 6-D feature space, able to express nonlinear decision boundaries (e.g. "hot AND small OR moderately-hot AND heavy-tailed") that a linear hyperplane over the raw 6 features cannot, without ever paying for the infinite dimensionality explicitly (adapted from `reports/checkpoint_report.tex`, "The dual and the kernel trick" / "The RBF kernel").

2.7 Decision function and probability calibration

The trained decision function, applied to a new (scaled) feature vector $\hat{x}$, is

$$f(\hat{x}) = \sum_{i \in \text{SV}} \alpha_i y_i k(x_i, \hat{x}) + b, \quad \hat{y} = \text{sign } f(\hat{x}),$$

a similarity-weighted vote of the stored support vectors — only points with $\alpha_i > 0$ (§2.4) need be kept, which is why the persisted checkpoint (`_state_dict` pickles the fitted SVC) is compact. Because `probability=True` is passed to SVC in `fit()`, sklearn additionally fits **Platt scaling** — a 1-D logistic regression $P(\text{fire} | x) = \sigma(a f(x) + c)$ on cross-validated decision values — giving the calibrated confidence field returned in every `FireAlert` (`predict_proba` in the code).

2.8 Implementation notes

- Feature order and NaN handling: `_frame_to_feature_vector` uses the *hottest* blob (`max(blobs, key=lambda b: b["max_temp"])`) among all connected components from `otsu_segment+extract_blobs`; a degenerate zero-variance blob produces NaN skew/kurtosis (matching scipy's convention) which is mapped to `0.0` via `np.nan_to_num` before scaling, since the scaler/SVM cannot accept non-finite input.

- `class_weight` is a constructor-time parameter (not a `fit()` argument) because sklearn fixes it at estimator construction; 'balanced' reweights the dual's box constraint per-class ($\alpha_i \leq C_{y_i}$ with class-dependent C) to counteract the fire-class rarity noted in `data/DATASET_NOTES.md`.
- `resolution_behavior = "invariant"`: because the feature vector is 6 scalar statistics rather than raw pixels, one checkpoint transfers across the MLX90640 and Waveshare sensor profiles without retraining.
- **CPU vs. GPU**: training and inference both run on CPU only. scikit-learn's SVC (and the SMO solver behind it, §2.3) has no GPU implementation, and even if it did, a 6-dimensional feature vector is far too low-dimensional for GPU parallelism to offer any benefit — the entire training set for this detector is subsampled to a few thousand examples per camera specifically because kernel-SVM training cost grows worse than linearly in the number of points (§2.8), not because of any per-example compute cost that a GPU would help with.

3. HOGSVMDetector — human_detection/hog_svm.py

3.1 Formulation

HOGSVMDetector treats human detection as **shape classification over a sliding window**: at every window position (and every scale in a pyramid), a Histogram-of-Oriented-Gradients (HOG) descriptor is computed and scored by a trained **linear** SVM (sklearn.svm.LinearSVC); windows scoring above `score_threshold` become candidate detections, and greedy NMS (non-maximum suppression — repeatedly keep the highest-scoring remaining box and discard every other box that overlaps it too much, until none are left) collapses overlapping candidates.

3.2 Derivation: the HOG descriptor

Let $I(x, y)$ denote the (per-window, normalized — see §3.5) pixel intensity. HOG proceeds in three stages, matching `skimage.feature.hog(patch, orientations=9, pixels_per_cell=cell_size, cells_per_block=block_size_cells, block_norm="L2-Hys")` exactly as called in `_hog_features`.

1. Gradient computation. At every pixel, horizontal and vertical gradients are computed (skimage's default is the centered finite difference $[-1, 0, 1]$):

$$G_x(x, y) = I(x+1, y) - I(x-1, y), \quad G_y(x, y) = I(x, y+1) - I(x, y-1),$$

from which the gradient magnitude and orientation are

$$m(x, y) = \sqrt{G_x^2 + G_y^2}, \quad \theta(x, y) = \arctan\left(\frac{G_y}{G_x}\right) \in [0^\circ, 180^\circ) \text{ (unsigned)}.$$

Thermal edges (a person silhouette against a cooler/warmer background) are exactly the kind of locally coherent gradient structure this captures — HOG's premise is that object **shape**, encoded in the local distribution of edge orientations, is a more scale/illumination-robust cue than raw intensity, which matters here because ambient ward temperature varies but silhouette shape does not.

2. Cell histogramming (orientation binning). The window is partitioned into non-overlapping cells of `cell_size` pixels (`self._cell_size`, default (2, 2) — deliberately tiny because a person silhouette at native MLX90640/Waveshare thermal resolution spans only a handful of pixels, so a single pixel is already a meaningful spatial unit). Within each cell, a histogram of `orientations = 9` bins is built over $[0^\circ, 180^\circ)$ by **soft-binning**: each pixel's gradient magnitude $m(x, y)$ is distributed (via linear interpolation) between its two nearest orientation bins in proportion to angular proximity, so a pixel does not have to fall exactly on a bin center to contribute:

$$h_{\text{cell}}(b) = \sum_{(x, y) \in \text{cell}} m(x, y) \cdot w_b(\theta(x, y)), \quad b = 1, \dots, 9,$$

where w_b is the (triangular) interpolation weight for bin b . This produces one 9-D vector per cell.

3. Block normalization. Cells are grouped into overlapping blocks of `block_size_cells` (default (2, 2) cells per block, `cells_per_block=(2,2)`), and the concatenated cell histograms within each block are normalized with **L2-Hys** — L2 normalize, clip to a maximum component value (0.2, skimage's default), then re-normalize:

$$v \leftarrow \frac{v}{\|v\|_2 + \epsilon}, \quad v \leftarrow \min(v, 0.2), \quad v \leftarrow \frac{v}{\|v\|_2 + \epsilon}.$$

Block-level (rather than whole-window) normalization makes the descriptor **locally** robust to contrast/illumination gradients across the window — one very hot sub-region does not wash out the gradient structure of a cooler sub-region elsewhere in the same window — which is exactly why the code additionally **pre**-normalizes each window to zero mean/unit variance (`_normalize_patch`) before computing HOG, so the detector is insensitive to the absolute ambient temperature of the scene as well as to local contrast.

The concatenation of all block-normalized histograms is the final HOG feature vector $X \in \mathbb{R}^d$, with d determined by (`window_size` / `cell_size`) and `block_size_cells` (`self._n_features`, recorded at fit time and validated at load time).

3.3 The linear SVM: primal objective and decision rule

The trained classifier is an `sklearn.svm.LinearSVC`, which — unlike `SVC` (§2) — solves the **primal** directly via the `liblinear` coordinate-descent solver rather than forming a Lagrangian dual. `LinearSVC` minimizes the L_2 -regularized (squared) hinge-loss objective

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^\top X_i + b))$$

directly over $(w, b) \in \mathbb{R}^{d+1}$ — the same **loss-plus-penalty** form used to motivate the dual in §2.2, but here it is the objective actually being minimized by the solver, not an equivalent reformulation of a QP that is then dualized. Because there is no kernel and no need to express the model as a weighted sum over training points, `liblinear` never forms Lagrange multipliers or a Gram matrix; it works directly in the d -dimensional primal weight space, which scales far better than kernel SVC when d is in the hundreds (as HOG feature vectors

are) and n (training windows) is large. **This is why the KKT/dual machinery of §2.3–2.5 is not the operative framework for HOGSVMDetector:** KKT conditions and strong duality remain true statements about problem (11) (it is still a convex QP-equivalent program with affine constraints once slacks are reintroduced), but the solver never constructs or exploits them — `liblinear` uses primal coordinate descent / trust-region Newton steps directly on (w, b) , so there are no α_i , no support vectors, and no kernel to discuss for this detector.

The fitted decision function is the plain affine score

$$f(X) = w^\top X + b, \quad \hat{y} = \text{sign } f(X),$$

which the code evaluates once per window as `self._svm.decision_function(features.reshape(1, -1))`, thresholds against `score_threshold`, and finally maps through a sigmoid $\sigma(f(X))$ purely to populate `Detection.score` with a quasi-probability in $(0, 1)$ — the sigmoid here is a monotonic rescaling for reporting, not a calibrated probability model (contrast with Platt scaling in §2.7, which is fit to data).

3.4 Why a linear kernel suffices/is preferred here

Three factors favor the linear model over the RBF approach used for fire (§2):

1. **Dimensionality.** HOG descriptors are high-dimensional (the window is deliberately sized from a physical person silhouette via `SensorProfile.physical_pixel_size_m`, then snapped to a cell-size multiple — typically producing d in the hundreds even at thermal resolution). In high dimensions, a rich-enough set of **linear** projections of the input (here, the 9-orientation-bin histogram) already encodes most of the discriminative shape information that a nonlinear kernel would otherwise have to reconstruct implicitly — the classic empirical finding (Dalal & Triggs, 2005) that motivated pairing HOG specifically with a linear SVM.
2. **No kernel trick needed.** Cover's theorem intuition: as feature dimensionality grows, a linear separator becomes increasingly likely to exist (or nearly exist) for a fixed number of training points, since the VC dimension of linear separators grows with d while the training-set size is fixed by how many labeled

person/background windows are available. 3. **Speed at inference.** The sliding-window multi-scale search (§3.6) evaluates the decision function at every window position and every pyramid scale — potentially thousands of evaluations per frame. A linear score $w^\top X + b$ is a single dot product per window; an RBF score would require a kernel evaluation against every stored support vector, which is asymptotically far more expensive precisely where this detector needs to be cheap.

3.5 Implementation notes: window sizing and normalization

- `_resolve_window_size` derives the pixel window from a **physical** target size (`physical_person_size_m`, default (0.9 m, 0.45 m)) and an assumed viewing distance via `SensorProfile.physical_pixel_size_m(distance_m)`, then `_snap_to_multiple` rounds down to an integer multiple of `cell_size` (required so HOG receives an integer cell grid) with a floor of $2 \times$ cell size per axis (so a 2×2 -cell block fits). This is why `resolution_behavior = "parameterized"`: the window/cell geometry auto-scales per sensor profile, but a checkpoint's learned (w, b) does not transfer between profiles because the feature dimensionality differs.
- `_normalize_patch` zero-means and unit-variances every window (both training patches and inference windows) before HOG is computed — this is what makes the detector insensitive to the **ambient** temperature of the room without needing to know it, complementing HOG's own block-level contrast normalization (§3.2).
- **CPU vs. GPU:** training and inference both run on CPU only. `LinearSVC`'s `liblinear` coordinate-descent solver (§3.3), like `SVC`'s SMO solver above, is a CPU-only implementation with no GPU code path — and the exhaustive multi-scale sliding-window search (§3.6) that dominates this detector's inference cost is itself a per-window Python/NumPy loop, not a batched tensor operation, so it would not benefit from a GPU even if the underlying linear-SVM scoring did.

3.6 Sliding-window multi-scale pyramid

`predict()` iterates over `pyramid_scales` (default (1.0,), but configurable to e.g. $\{0.75, 1.0, 1.25, 1.5\} \times$, as evaluated in `reports/human_detection_report.tex`), resizing the frame at each scale (`cv2.resize`, `INTER_LINEAR` when upscaling / `INTER_AREA` when downscaling) and sliding the **fixed-size** detection window over the resized frame with `stride`. A window at scale s , positioned at (x_0, y_0) in the resized frame, maps back to the original frame's coordinates by dividing by s : $(x_0/scale, y_0/scale, ww/scale, wh/scale)$. Running the same fixed window over multiple resized copies of the image is algorithmically equivalent to running variable-size windows over one fixed image, but is simpler to implement given a HOG extractor tied to one cell/block geometry; it lets the detector match people who appear larger or smaller than the training window's implicit physical scale (e.g., closer/farther from the sensor) — at the empirically documented cost of somewhat more false positives per `reports/human_detection_report.tex`'s pyramid ablation, since more windows are scored in total. All boxes across all scales are pooled and a single greedy NMS pass (`_greedy_nms`, `IoU threshold nms_iou_threshold`) removes duplicate detections of the same person.

4. MobileNetSSDDetector — human_detection/mobilenet_ssd.py, mobilenet_ssd_model.py, mobilenet_ssd_anchors.py

4.1 Formulation

MobileNetSSDDetector is a single-stage, anchor-based deep detector (SSD — Single Shot (multibox) Detector, Liu et al., 2016: predict boxes and classes directly from feature-map cells in one forward pass, with no separate region-proposal stage before classification — style) built on a small MobileNet backbone (MicroMobileNet), run directly on the native sensor resolution (no upsampling to 300×300). Two feature maps F1 (fine) and F2 (coarse) are tapped from the backbone; each spatial cell of each map predicts, for a fixed set of K anchor boxes anchored at that cell, 4 box-regression offsets and $\text{num_classes} = 2$ (background/person) class logits.

4.2 Depthwise-separable convolutions (backbone)

Each DepthwiseSeparableBlock factors a standard convolution into two cheaper steps, exactly as implemented:

$$\text{depthwise: } y_c(x, y) = \sum_{(dx, dy) \in \mathcal{K}} w_c(dx, dy) x_c(x + dx, y + dy) \quad (\text{one filter per input channel } c),$$

$$\text{pointwise: } z_{c'}(x, y) = \sum_c u_{c', c} y_c(x, y) \quad (1 \times 1 \text{ conv, channel mixing only}).$$

A standard 3×3 convolution from C_{in} to C_{out} channels costs $3 \cdot 3 \cdot C_{\text{in}} \cdot C_{\text{out}}$ multiply-accumulates per output pixel; the depthwise+pointwise factorization costs $3 \cdot 3 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}}$, a ratio of $(1/C_{\text{out}}) + (1/9)$ — roughly an order of magnitude cheaper for the channel counts used here (adapted from reports/checkpoint_report.tex, "Why depthwise-separable convolutions"). This is what keeps MicroMobileNet cheap enough to run per-frame at the target frame rate.

4.3 Anchor generation

generate_anchors places, at every cell (i, j) of a feature map of size $f_h \times f_w$ over an input of size $i_h \times i_w$, $K = |\text{aspect_ratios}|$ anchors centered at the cell's projected input-pixel location:

$$(cx, cy) = ((j + \frac{1}{2}) \cdot \frac{i_w}{f_w}, (i + \frac{1}{2}) \cdot \frac{i_h}{f_h}), \quad w = \text{anchor width}, \quad h = \frac{w}{\text{aspect ratio}},$$

for each aspect ratio in $\text{aspect_ratios} = (1.0, 0.5, 0.33)$ (square, 2×-tall, 3×-tall — matching a standing person's silhouette). F1 and F2 use different anchor_widths per profile (e.g. Waveshare: 10 px at F1, 16 px at F2), so the two feature maps jointly cover a range of apparent person sizes without an image pyramid: F1's finer grid and smaller anchors catch distant/small people, F2's coarser grid and larger anchors catch close/large ones (MicroMobileNetSSD.build_anchors concatenates both levels' anchors into one $(N_{\text{anchors}}, 4)$ buffer in (cx, cy, w, h) form).

4.4 IoU-based anchor matching (positive/negative assignment)

For a ground-truth box set $\{g_k\}$ and anchor set $\{a_m\}$, pairwise_iou computes, in corner form (x_1, y_1, x_2, y_2) derived from (cx, cy, w, h) via cxcywh_to_xyxy,

$$\text{IoU}(a, g) = \frac{|a \cap g|}{|a \cup g|} = \frac{\max(0, \min(x_2^a, x_2^g) - \max(x_1^a, x_1^g)) \cdot \max(0, \min(y_2^a, y_2^g) - \max(y_1^a, y_1^g))}{|a| + |g| - |a \cap g|}.$$

IoU (Intersection over Union — the shared area of two boxes divided by their combined area; 1.0 for identical boxes, 0 for non-overlapping ones) is the standard measure of how well one box matches another.

match_anchors_to_targets then assigns, for each anchor m , a label via the standard SSD rule implemented exactly as follows:

1. Every anchor's best-matching ground truth is $g^*(m) = \arg \max_k \text{IoU}(a_m, g_k)$, with score $\text{IoU}^*(m)$.
2. Anchor m is **positive**, matched to $g^*(m)$, if $\text{IoU}^*(m) \geq \text{iou_pos_threshold}$ (0.5).
3. Anchor m is **negative** (background) if $\text{IoU}^*(m) < \text{iou_neg_threshold}$ (0.4); anchors in $[0.4, 0.5)$ are **ignored** (contribute to neither loss term).
4. **Forced positive override**: for every ground-truth box g_k , the single anchor $\arg \max_m \text{IoU}(a_m, g_k)$ is *always* forced positive regardless of its raw IoU — this guarantees every labeled person has at least one responsible anchor, even a small/oddly-shaped person whose best IoU with any anchor never reaches 0.5.

4.5 The multi-task SSD loss

Given the matching of §4.4, `ssd_loss` computes two terms per training sample, matching the code exactly.

Localization loss (positives only). For each positive anchor m matched to $g^*(m)$, the target is the **encoded offset** of the ground-truth box relative to the anchor (not the absolute box) — this is what `encode_boxes` computes:

$$\Delta_{cx} = \frac{cx_g - cx_a}{w_a \cdot v_{xy}}, \quad \Delta_{cy} = \frac{cy_g - cy_a}{h_a \cdot v_{xy}}, \quad \Delta_{\log w} = \frac{\log(w_g/w_a)}{v_{wh}}, \quad \Delta_{\log h} = \frac{\log(h_g/h_a)}{v_{wh}},$$

with the standard SSD variance scales $v_{xy}=0.1$, $v_{wh}=0.2$ (BBOX_VARIANCE_XY, BBOX_VARIANCE_WH) rescaling the raw offsets so all four regression targets have comparable magnitude/gradient scale during training — the network's box head predicts these four numbers directly, and `decode_boxes` is the exact algebraic inverse used at inference (§4.7). The loss on these offsets is **Smooth-L1** (Huber),

$$\text{SmoothL1}(e) = \frac{1}{2}e^2 \text{ if } |e| < 1, \text{ else } |e| - \frac{1}{2}, \quad \mathcal{L}_{\text{loc}} = \sum_{m \text{ positive}} \text{SmoothL1}(\text{pred}_m - \Delta(g^*(m), a_m)),$$

implemented as `F.smooth_l1_loss(..., reduction="sum")`. Smooth-L1 is quadratic (like squared error) for small residuals — precise fine-tuning near the target — but linear for large residuals, so a handful of badly-matched anchors early in training cannot produce the exploding gradients that pure squared error would (adapted from `reports/checkpoint_report.tex`).

Classification loss (positives + hard negatives). Every anchor's binary target is $c_m = 1$ if matched positive, 0 otherwise (background):

$$\mathcal{L}_{\text{cls}} = \sum_{m \in \text{keep}} \text{CE}(\text{cls_preds}_m, c_m), \quad \text{CE}(\ell, c) = -\log \text{softmax}(\ell)_c,$$

CE (cross-entropy — how "surprised" the model's predicted probability distribution is by the true label; near zero when confident and correct, large when confident and wrong) is the standard loss for classification.

implemented as `F.cross_entropy(..., reduction="sum")` over the anchors selected by hard-negative mining (§4.6).

Total loss, normalized by the number of positive anchors in the batch (so the loss scale is independent of how many people are in a given frame):

$$\mathcal{L} = \frac{\mathcal{L}_{\text{cls}}}{\max(N_{\text{pos}}, 1)} + \lambda \cdot \frac{\mathcal{L}_{\text{loc}}}{\max(N_{\text{pos}}, 1)}, \quad \lambda = \text{loc_loss_weight} = 1.0,$$

exactly the `cls_loss + loc_loss_weight * loc_loss` computed in `ssd_loss`. This is a genuinely **multi-task** loss because it trains the two conv heads (box_head, cls_head) jointly, sharing the backbone — the classification term teaches *which* anchors contain a person, the localization term teaches *where* the box should sit relative to the matched anchor.

4.6 Hard-negative mining

With `num_classes=2` and typically hundreds to over a thousand anchors (720 for MLX90640, 1110 for Waveshare, per the module docstring) but usually only a handful of people per frame, the vast majority of anchors are background. Training on *all* negatives would let the trivial "always predict background" solution dominate the loss. `hard_negative_mining` instead:

1. Computes each anchor's log-softmax background score $\log p(\text{class}=0 | \text{anchor})$.
2. Restricts attention to anchors labeled negative (`matches == NEG_GT`).
3. Keeps only the `n_neg_keep = min(neg_pos_ratio * n_pos, n_negatives_available)` **hardest** negatives, i.e. those with the *lowest* background log-probability — the negatives the network is currently most confidently (and wrongly) calling "person-like."
4. Returns `pos_mask | keep_neg` as the boolean mask fed into $\mathcal{L}_{\text{cls}}$'s sum.

With `neg_pos_ratio = 3.0` (the constructor default), this fixes the classification loss's positive:negative ratio at 1:3 regardless of how imbalanced the raw anchor population is, concentrating gradient signal on the negatives that are actually informative (near-misses) rather than the overwhelming majority of easy true negatives far from any person.

4.7 Inference: decoding and NMS

At inference, `decode_predictions` inverts the encoding of §4.5 (`decode_boxes`, the algebraic inverse of `encode_boxes`) to recover absolute (cx, cy, w, h) boxes from the network's raw offset predictions, converts to corner form, applies `softmax` to the class logits to get calibrated-in-training-objective probabilities, and keeps boxes with $P(\text{person}) > \text{score_threshold}$ (0.5 default). Because neighboring anchors and both feature-map levels typically all fire on the same physical person, `nms_xyxy` — the same greedy algorithm as `_greedy_nms` in §3.6 — then collapses duplicate boxes: sort by score, keep the top box, discard every remaining box with $\text{IoU} > \text{nms_iou_threshold}$ (0.3) against it, and repeat.

4.8 Implementation notes

- `resolution_behavior = "fixed"`: because the backbone's channel/stride schedule and the anchor grid's absolute pixel geometry are baked into `BackboneConfig/SSDConfig` per profile (`MLX90640_BACKBONE/_SSD` vs. `WAVESHARE_BACKBONE/_SSD`), a checkpoint trained for one sensor's (H, W) cannot be loaded for the other — two independent checkpoints are required, unlike the resolution-invariant `FireSVMDetector` (§2) or the parameterized `HOGSVMDetector` (§3).
- Training uses per-frame z-score normalization (`_augment/_normalize: (x - μ)/(σ + 10-6)`) plus random horizontal flip and small translation augmentation (with box coordinates transformed accordingly) — see `MobileNetSSDDetector._augment`.
- Adam optimizer with gradient-norm clipping (`clip_grad_norm_(max_norm=5.0)`) is used for the (non-convex) network training, in sharp contrast to the convex QP of §2 — this is a fundamentally different optimization regime (stochastic gradient descent on a non-convex loss surface, no KKT/global-optimality guarantee) than the SVM sections.
- **CPU vs. GPU**: training and inference run on GPU when one is available (`torch.cuda.is_available()`), falling back to CPU otherwise. Unlike the two SVM-based

detectors above, this is a batched neural network with many small convolutions over a full feature map — exactly the kind of workload that benefits from a GPU's parallel matrix/tensor units, so device choice has a real, measurable effect on this detector's inference latency (see the report's ONNX/quantization results for a concrete before/after comparison).

5. GeometricContactDetector — `contact_detection/geometric.py`, `multi_view/homography.py`, `multi_view/fusion.py`

5.1 Formulation

`GeometricContactDetector` is a deterministic, non-learned pipeline: it projects each camera's 2-D bounding-box detections onto a shared floor plane via a calibrated homography, fuses cross-camera projections into unique "actor" nodes, and flags contact when two actors' floor-plane positions fall within a fixed distance δ .

5.2 Homography via DLT (Direct Linear Transform — solve for an unknown transformation by writing its defining equations as one big linear system and solving that system directly, rather than iterating) + SVD (Singular Value Decomposition — factor a matrix into rotate/scale/rotate pieces that expose exactly which directions the matrix stretches most and least; used below to find the least-stretched direction)

Setup. A planar homography $H_k \in \mathbb{R}^{3 \times 3}$ maps a camera- k image point (u, v) (homogeneous $[u, v, 1]^\top$) to a floor-plane world point (X_w, Y_w) (homogeneous $[X_w, Y_w, 1]^\top$, up to scale) whenever the imaged points lie on a single plane (here, $Z = 0$, the floor):

$$w [X_w, Y_w, 1]^\top \cong H_k [u, v, 1]^\top.$$

Deriving the DLT linear system. Write H_k row-wise as $h_1^\top, h_2^\top, h_3^\top$ (rows of H_k) so that $X_w = (h_1^\top p)/(h_3^\top p)$, $Y_w = (h_2^\top p)/(h_3^\top p)$ with $p = [u, v, 1]^\top$. Clearing denominators,

$$h_1^\top p - X_w h_3^\top p = 0, \quad h_2^\top p - Y_w h_3^\top p = 0,$$

which, written out in the 9 unknowns $h = \text{vec}(H_k)$, are exactly the two rows the code assembles per correspondence:

$$\text{row}_1 = [-u, -v, -1, 0, 0, 0, uX_w, vX_w, X_w] \cdot h = 0, \quad \text{row}_2 = [0, 0, 0, -u, -v, -1, uY_w, vY_w, Y_w] \cdot h = 0.$$

Each of the $N \geq 4$ calibration correspondences $(u_i, v_i) \leftrightarrow (X_{w,i}, Y_{w,i})$ (heated targets at known floor positions — "Hot-Point Calibration," since checkerboard patterns have no thermal contrast) contributes both rows, stacking into a $2N \times 9$ homogeneous system $Ah = 0$.

Why SVD gives the solution. $Ah = 0$ has the trivial solution $h = 0$, which is useless; the meaningful constrained problem is $\min_h \|Ah\|^2$ subject to $\|h\| = 1$ (fixing the scale ambiguity inherent to homogeneous coordinates — H_k and λH_k represent the same projective map). Writing $A = U\Sigma V^\top$ (the SVD), $\|Ah\|^2 = \|U\Sigma V^\top h\|^2 = \|\Sigma V^\top h\|^2$ (since U is orthogonal), which is minimized over unit-norm h by setting h equal to the right-singular vector corresponding to Σ 's **smallest** singular value — because $\Sigma V^\top h$ then has all its energy concentrated in the smallest diagonal entry of Σ . This is exactly `_dlt_solve's` `Vt[-1]` (the last row of $V^\top$, i.e. the smallest-singular-value right singular vector of `np.linalg.svd(A)`), reshaped to 3×3 and rescaled so $H[2, 2] = 1$ (an arbitrary but standard normalization removing the remaining scale ambiguity). With more than the minimal 4 correspondences, this is simultaneously the **least-squares** solution to the over-determined homogeneous system — the SVD approach degrades gracefully from an exact solve to a best-fit as N grows, which is why the docstring recommends more than the minimum 4 points for accuracy.

5.3 Foot-point projection

Given a detection's bounding box (x, y, w, h) (top-left corner convention), the code approximates its floor contact point by the **bottom-center** of the box,

$$P_{\text{foot}} = (x + \frac{w}{2}, y + h)$$

(`Detection.foot_point`) — physically justified because a standing or seated person's bounding box, in a top/side thermal view, has its base at the floor and its horizontal center roughly aligned with the body's vertical axis, so the bottom-center is the best single-point proxy for "where the

person is touching the ground" available from a 2-D box alone. `project_foot_point` then applies Eq. (12) directly: $p_{\text{world}} = H_k [u, v, 1]^\top$, followed by the homogeneous divide $X_w = p_{\text{world}}[0]/p_{\text{world}}[2]$, $Y_w = p_{\text{world}}[1]/p_{\text{world}}[2]$ (guarded against $|w| < 10^{-9}$, i.e. points that map to infinity).

5.4 Multi-view fusion: clustering and validation

`fuse_detections` (`$multi_view/fusion.py`) projects every camera's detections to world points, then:

1. **Greedy single-linkage clustering** (`_greedy_cluster`): a world point joins the first existing cluster containing any member within `epsilon_m` (ϵ , default 0.5 m) of it; otherwise it starts a new cluster. This groups projections of the **same** physical person seen from different cameras. 2. **Validation** (`_validate_cluster`), applying the physical prior that a genuine person should be corroborated by more than one camera:

- $N = 1$ (single-camera): discarded — no cross-camera corroboration, so a lone projection cannot be distinguished from a false detection or a projection error.
- $N = 2$: accepted iff the pairwise distance is $< \epsilon$ (consistency check).
- $N = 3$: **outlier removal** — if exactly one point is $> \epsilon$ from **both** others while the remaining pair is mutually $< \epsilon$, that point is dropped and the consistent pair is kept; if all three are mutually consistent, all three are kept; otherwise (no consistent pair) the cluster is discarded entirely.
- $N > 3$: all points kept (already enforced to be within ϵ pairwise-transitively by the clustering step).

3. Each validated cluster's centroid $(\bar{X}, \bar{Y}) = ((1/n)\sum x_i, (1/n)\sum y_i)$ becomes one `ActorPosition`, with confidence the mean of its members' detector confidences and `source_camera_ids` recording which cameras contributed.

5.5 The Euclidean-distance contact rule

Once unique actors' floor positions are known, `GeometricContactDetector.predict` computes, for every actor pair (i, j) ,

$$D_{ij} = \sqrt{(X_i - X_j)^2 + (Y_i - Y_j)^2}, \quad \text{contact}(i, j) \Leftrightarrow D_{ij} < \delta_m$$

(`delta_m`, default 0.5 m) — a direct thresholded Euclidean distance on the projected floor plane.

5.6 Why a simple threshold is a valid decision rule here

Physical contact between two people is, almost by definition, a statement about the distance between their bodies falling below the scale of a human body — two people cannot be "touching" while their centroids (or, here, foot-points) are meters apart. Once the earlier pipeline stages (detection → foot-point extraction → homographic projection → cross-camera fusion) have already converted image-plane pixel coordinates into a single, metric, common floor-plane coordinate system, "distance below a body-scale threshold" is not an approximation of the physical contact criterion — it *is* the physical contact criterion, restated in the coordinate system the pipeline has arranged to make it directly checkable. This is why no learned classifier is needed at this final stage (`is_trainable = False`; `fit()` is a no-op): the earlier geometric/homographic stages have already done the work of turning "contact" into a metric-distance question, and a threshold in metric units is the natural (and, given a correctly calibrated homography, essentially exact) answer to that question. The two free parameters — ε (clustering/consistency) and δ (contact) — are tuned *independently* because they answer different questions: ε bounds the expected *projection noise* of the homography (how far apart two projections of the *same* person's foot-point can legitimately land due to calibration error), while δ bounds the physical extent of *two different* people's bodies when in contact; conflating them would either over-merge distinct nearby people (if ε were set to the larger δ -scale value) or under-merge noisy projections of one person (if δ were set to the smaller ε -scale value).

6. MVSTGCNDetector —

`contact_detection/mv_stgcn.py`

6.1 Formulation

`MVSTGCNDetector` extends the geometric front-end of §5 with a learned spatiotemporal classifier: a sliding window of $T=16$ timesteps of actor-graphs is fed to a small Graph Convolutional Network (`_ThermalSTGCN`) that outputs a 2-class (contact/no-contact) probability, gated by a persistence rule before alerting.

6.2 Node features (7-D)

For each of up to `max_actors` ($N=5$ default) actors at a given timestep, `_actors_to_node_features` builds the 7-D vector

$$f = [X_{\text{norm}}, Y_{\text{norm}}, v_x, v_y, T_{\text{max, norm}}, T_{\text{mean, norm}}, \text{area}] \in \mathbb{R}^7,$$

with world position normalized by an assumed room scale ($X/5$, $Y/5$, "room ≈ 5 m"), velocity from the per-camera Kalman tracker's constant-velocity state (scaled by $1/10$), and thermal statistics scaled by $1/100$. Padded (absent) actor slots get an all-zero feature row and `mask=0`; the mask is threaded through every subsequent computation so padding never influences the graph convolution or the final pooled representation.

6.3 Adaptive (Gaussian-distance) graph adjacency

For a batch of N nodes at one timestep with world positions $\{p_n\}$, `_AdaptiveGCNLayer.forward` computes the squared pairwise distance matrix

$$d_{ij}^2 = \|p_i - p_j\|^2$$

(computed once per timestep in `_ThermalSTGCN.forward` via broadcasted subtraction over the flattened $(B \cdot T, N, 2)$ position tensor) and derives a **row-stochastic, distance-decaying adjacency**:

$$A_{ij} = \frac{\exp(-d_{ij}^2/\sigma^2)}{\sum_{j'} \exp(-d_{ij'}^2/\sigma^2)} = \text{softmax}_j \left(-\frac{d_{ij}^2}{\sigma^2} \right),$$

exactly `F.softmax(-dist2/sigma2, dim=2)` in the code, with padded columns masked to $-\infty$ before the softmax so they receive zero weight. This is a **Gaussian-kernel graph**: nearby actors (small d_{ij}) get large edge weight, distant actors

get exponentially suppressed weight — the natural continuous relaxation of "connect actors that are near each other," parameterized by the learnable/fixed bandwidth σ ($\text{sigma}=1.0$ default in the layer). Unlike a fixed geometric graph (e.g. connect-if- $d < \delta$, as in §5.5), this adjacency is **soft and differentiable**, allowing gradient-based training to shape how strongly proximity translates into information flow.

6.4 Propagation rule

Self-loops are added ($\hat{A} = A + I$) so every node retains its own features in the aggregation, then **symmetric-style row normalization** is applied via the node degree $d_i = \sum_j \hat{A}_{ij}$:

$$\bar{A}_{ij} = \frac{\hat{A}_{ij}}{d_i}, \quad (\text{the code's } A_{\text{norm}} = \hat{A}/\text{deg, i.e. } D^{-1}\hat{A}, \text{ a row-normalized rather than symmetric } D^{-1/2}\hat{A}D^{-1/2} \text{ variant}),$$

and one GCN layer computes

$$H^{(\ell+1)} = \text{ReLU}(\bar{A} H^{(\ell)} W^{(\ell)}), \quad H^{(0)} = f \text{ (the 7-D node features),}$$

which is the standard Kipf–Welling graph-convolution propagation rule (`support = A_norm @ x` then `W(support)` then `ReLU`, masked by the actor-presence mask so padded nodes stay zero), stacked `n_gcn_layers` times (default 2). Equation (15) is the exact graph analogue of a normal feed-forward layer $H^{(\ell+1)} = \text{ReLU}(H^{(\ell)} W^{(\ell)})$, with the crucial difference that each node's pre-activation is first **averaged with its (adjacency-weighted) neighbors** before the linear map — this is what lets a node's updated representation depend on nearby actors' state, which is the entire point of using a graph rather than treating each actor independently.

6.5 Spatiotemporal structure over the $T=16$ window

The full architecture is genuinely **spatiotemporal**, but not via explicit temporal graph edges: at each of the $T=16$ timesteps in the sliding window, the N actors form a **fully-connected spatial graph** (every pair gets an adjacency weight from Eq. 14; there is no discrete "connect/don't connect" spatial edge decision — the Gaussian kernel makes every pair weakly-to-strongly connected depending on distance). The **temporal** dimension is handled not by an explicit temporal adjacency but by **weight-sharing across time combined with global pooling**: `_ThermalSTGCN.forward` reshapes $(B, T, N, \cdot) \rightarrow (B \cdot T, N, \cdot)$, applies the **same** GCN layers independently to every timestep's spatial graph (this is effectively T chained/replicated spatial-only graphs, not a spatiotemporal graph with per-node temporal self-edges — the "temporal chain" connecting the same actor across consecutive timesteps is realized only implicitly, through the Kalman-filtered velocity feature v_x, v_y carried in each node's own feature vector, and explicitly through the final pooling step), then reshapes back to (B, T, N, hidden) and performs a ****masked global average pool over both T and N ****:

$$z = \frac{\sum_{t,n} \text{mask}_{t,n} \cdot H_{t,n}^{(\text{final})}}{\sum_{t,n} \text{mask}_{t,n}} \in \mathbb{R}^{\text{hidden_dim}},$$

which the classifier head (`Linear(hidden_dim, 32) → ReLU → Linear(32, 2)`) maps to 2 class logits, softmax-normalized to a contact probability. The sliding window of $T=16$ gives the GCN's temporal context — a single instant's actor graph cannot capture **approach and interaction over time**, but 16 consecutive fully-connected spatial graphs, each carrying velocity, jointly encode a short spatiotemporal trajectory even though the graph convolution itself only mixes information within a timestep.

6.6 Implementation notes

- `resolution_behavior = "invariant"`: all graph computation operates on world-space (metric) coordinates from the homographic front-end (§5), so — like `GeometricContactDetector` — the same trained model works across sensor profiles.

- The persistence gate (`conf_threshold`, `persistence_frames`) requires the softmax contact probability to exceed threshold for `persistence_frames` *consecutive* `predict()` calls before actually alerting, suppressing single-frame noise in the GCN's output — a rule applied on top of, not inside, the network.
- Training minimizes `nn.CrossEntropyLoss` (optionally class-weighted for the contact-minority imbalance, per `data/DATASET_NOTES.md`) via Adam, i.e. ordinary non-convex SGD-family training — the KKT/duality framework of §2 does not apply to this model for the same reason it does not apply to §4's SSD network: the objective is a non-convex function of the network weights.
- **CPU vs. GPU:** as with `MobileNetSSDDetector` (§4.8), training and inference run on GPU when available for the same reason — batched tensor operations through the GCN layers benefit from parallel hardware — falling back to CPU otherwise.

7. ThermoX3DDetector — `contact_detection/thermo_x3d.py`

7.1 Formulation

`ThermoX3DDetector` is the pixel-based, geometry-free failsafe contact detector: it processes raw (T, H, W) thermal volumes from each camera directly through a small factorized 3-D CNN, with **no bounding boxes, no homography, and no per-camera fusion** — a design response to the documented failure mode of §5–6 where close contact causes two people's bounding boxes to visually merge, leaving the box-based detectors with a single blob and no way to represent "two actors in contact" (`mv_stgcn.py/geometric.py` both depend on the upstream `HumanDetector` correctly separating actors; `Thermo-X3D` does not).

7.2 3-D convolution as a generalization of 2-D convolution

A standard 2-D convolution on an image $I(y, x)$ with kernel w of spatial extent (k_h, k_w) produces

$$O(y, x) = \sum_{dy = -k_h/2}^{k_h/2} \sum_{dx = -k_w/2}^{k_w/2} w(dy, dx) I(y + dy, x + dx).$$

A **3-D convolution** simply extends the kernel and the sliding window along a third axis — here, time t — over a volume $V(t, y, x)$ with a kernel of extent (k_t, k_h, k_w) :

$$O(t, y, x) = \sum_{dt = -k_t/2}^{k_t/2} \sum_{dy} \sum_{dx} w(dt, dy, dx) V(t + dt, y + dy, x + dx).$$

Equation (16) reduces exactly to the 2-D case when $k_t = 1$ (as the `_MicroX3DStream.stem`'s `kernel_size=(1,3,3)` does) — a 3-D convolution is a strict generalization of a 2-D convolution, obtained by allowing the receptive field to span multiple *frames*, not just multiple *pixels within one frame*; this is what lets the network respond to genuinely spatiotemporal patterns (two hot blobs' shapes merging over several frames) rather than only to any single frame's static appearance.

7.3 (2+1)D factorization

Rather than a full 3-D kernel $w(dt, dy, dx)$ with $k_t \cdot k_h \cdot k_w$ free parameters per input/output channel pair, `_FactorizedResBlock` **factorizes** each block into a purely spatial 3-D convolution (temporal extent 1) followed by a purely temporal 3-D convolution (spatial extent 1), matching the code precisely:

spatial: `Conv3d(kt = 1, kh = 3, kw = 3, stride = (1, s, s))` → BN → ReLU,

temporal: `Conv3d(kt = 3, kh = 1, kw = 1, stride = 1)` → BN → ReLU,

BN (batch normalization — rescale each channel's activations to roughly zero mean/unit variance using the current mini-batch's own statistics) stabilizes and speeds up training of deep stacks like this one.

i.e. $w(dt, dy, dx) \approx w_{\text{sp}}(dy, dx) \cdot w_{\text{temp}}(dt)$ — a rank-1-like separable approximation of the full 3-D kernel. This factorization: (i) reduces parameters/FLOPs from $O(k_t k_h k_w)$ to $O(k_h k_w + k_t)$ per channel pair, (ii) inserts an extra BN+ReLU nonlinearity between the spatial and temporal steps (more expressive than one linear 3-D conv for the same or lower parameter count), and (iii) — since every individual op is now either a plain 2-D-shaped convolution ($1 \times 3 \times 3$) or a plain

1-D-shaped convolution ($3 \times 1 \times 1$) — maps well onto CPU inference kernels that are heavily optimized for 2-D convolution, which matters for the edge (Raspberry Pi CPU) deployment target (adapted from `reports/checkpoint_report.tex`,

"(2+1)D-factorized"). Note the current implementation's window is `(ThermoX3DDetector.__init__(T=16))` with per-corpus global normalization (§7.5), differing from the $T=5$ /rolling-p25-background configuration documented for the deployed "T5v2" checkpoint variant in `reports/checkpoint_report.tex` — the derivation above is architecture-level and applies to either T .

Each residual block also applies a **skip connection** — identity if channel count and stride are unchanged, else a $1 \times 1 \times 1$ projection conv + BN — added after the (attention-gated) spatial+temporal path and before a final ReLU: `out = ReLU(skip(x) + attention(temporal(spatial(x))))`, the standard ResNet-style residual formulation lifted to 3-D, which keeps gradients well-behaved through the three stacked blocks (24→48→96 channels, spatial stride 2 at blocks 2 and 3).

7.4 CBAM (Convolutional Block Attention Module — learn two small, cheap gates, one over channels and one over space, that reweight a feature map by how relevant each part is, rather than treating every channel/pixel as equally important)-style thermal attention

Before the residual sum, `_ThermalAttention` reweights the block's output feature map using channel and spatial gates, matching the code:

Channel: $M_c = \sigma(\text{MLP}(\text{AvgPool}_{T,H,W}(X)) + \text{MLP}(\text{MaxPool}_{T,H,W}(X))) \in (0, 1)^C$, $X' = X \cdot M_c$,

Spatial: $M_s = \sigma(\text{Conv}_{1 \times 7 \times 7}([\text{AvgPool}_C(X'); \text{MaxPool}_C(X')]))$, $X'' = X' \cdot M_s$,

with the same 2-layer-MLP-with-reduction-4 channel gate and the same 2-channel-input 7×7 spatial gate as the CBAM formulation (Woo et al., 2018), adapted to 3-D pooling (over (T, H, W) for the channel gate, over the channel axis for the spatial gate). Using *both* average- and max-pooling in the channel gate lets the network respond both to a region's overall thermal energy (average) and to its single hottest voxel (max) — appropriate for a signal (contact) where both "two warm blobs merging" (average-detectable) and "a sharp new hot edge

appearing where two bodies meet" (max-detectable) are informative. This attention is $O(C) + O(HW)$ per block (a shared small MLP over pooled statistics, plus one 7×7 conv over an already-pooled 2-channel map) — categorically cheaper than quadratic-cost self-attention, since it never forms an $N \times N$ token-similarity matrix (adapted from `reports/checkpoint_report.tex`, "Why attention here is cheap").

7.5 Global normalization

Unlike the per-frame normalization used by `MobileNetSSDDetector` (§4.8), `ThermoX3DDetector.fit()` computes **corpus-wide** normalization statistics once, over every pixel of every frame in the training set:

$$\mu_{\text{global}} = \frac{1}{M} \sum_{k=1}^M v_k, \quad \sigma_{\text{global}} = \sqrt{\frac{1}{M} \sum_k (v_k - \mu_{\text{global}})^2} + 10^{-6},$$

(`self._global_mean`, `self._global_std`, over the concatenation of all `frame.data.ravel()` across the training set) and applies $(x - \mu_{\text{global}}) / \sigma_{\text{global}}$ identically at training and inference (`_normalise`, persisted in `_state_dict/_load_state_dict` so inference exactly reproduces the training distribution). This is the correct choice specifically because the network's job is to detect a spatiotemporal *pattern* (bodies merging/interacting) rather than to be invariant to each individual clip's own brightness — per-frame normalization would erase genuine cross-frame temperature differences (e.g., a person's silhouette warming or two bodies' thermal signatures blending as they make contact) that are exactly the signal this detector is trying to learn; a single fixed global affine map preserves those relative differences throughout a T-frame volume.

7.6 Late fusion and the softmax classification head

Each camera stream is encoded **independently** by its own `_MicroX3DStream` (three separate instances in `nn.ModuleList`, no shared weights) down to a 128-D vector via `AdaptiveAvgPool3d(1)` (**global average pooling** — collapsing the entire (T, H, W) volume to one value per channel, appropriate because contact is a spatially/temporally *distributed* event rather than one localized peak, unlike a use-case where global max-pooling would be preferred; adapted from `reports/checkpoint_report.tex`, "Why global average pool at the head") followed by `Linear(96, 128)`. The three 128-D descriptors are concatenated (**late fusion**, $v_{\text{final}} = [v_1; v_2; v_3] \in \mathbb{R}^{384}$) and passed through the classifier head:

$$z = W_2 \text{ReLU}(W_1 v_{\text{final}} + b_1) + b_2 \in \mathbb{R}^2, \quad p(\text{contact}) = \text{softmax}(z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}},$$

```
matching                self.head                =  
Sequential(Linear(384, 64),                ReLU(),  
Linear(64, 2)) followed by F.softmax(logits,  
dim=1)[0, 1] in _run_x3d_inference. Late fusion  
(concatenate-then-classify, rather than e.g. averaging  
per-camera probabilities) lets the classifier learn cross-camera  
relationships — e.g. "contact visible from camera 1 but not  
camera 2" is a different pattern than "contact visible from all  
three" — that a simple probability average would discard.
```

7.7 Implementation notes

- `resolution_behavior = "fixed"`: the spatial dimensions (H, W) of every `Conv3d` layer's receptive field interact with the sensor's native resolution through the two spatial-stride-2 blocks, so — as with `MobileNetSSDDetector` — one checkpoint per sensor profile is required (`sensor_profile.resolution` fixes `self._input_h, self._input_w` at construction).
- The rolling per-camera buffer (`deque(maxlen=T)`) implements the "fixed-step sampling" described in the module docstring: `predict()` is called once per incoming frame and always evaluates the network on the most recent T buffered frames, decoupling the network's temporal window from the capture frame rate.

- Training minimizes `nn.CrossEntropyLoss` (optionally class-weighted) via Adam — ordinary non-convex network training, as in §4 and §6; there is no KKT/duality argument for this model, only the standard (heuristic, not globally-guaranteed) convergence behavior of gradient descent on a non-convex loss surface.
- **CPU vs. GPU**: training and inference run on GPU when available, falling back to CPU otherwise. Of the three neural detectors in this report, this is the one where GPU acceleration matters most in practice: 3-D convolutions over full (T, H, W) volumes are substantially more compute-intensive per forward pass than MobileNet-SSD's 2-D backbone or MV-STGCN's small per-timestep graph, so this detector's chunked full-corpus training run is the most sensitive of the five to compute-device throughput.

8. RBTCT — Rule-Based Temporal Contact Tracker — `contact_detection/rbtct.py`

8.1 Formulation

RBTCT (historically "config-D") is the strongest *homography-free* contact detector found in this project, and the only one that uses **no contact-specific training whatsoever** — it consumes a person detector (§4) and a thermal residual, and every remaining parameter is a threshold tuned on a validation split. It exists because both homography-based contact detectors (§5, §6) inherit a measurement-noise problem: the self-calibrated homography's accuracy ($\approx \pm 15$ px) is nearly as large as the contact decision threshold itself ($\delta \approx 18$ px), so inter-actor distance in the floor plane is barely more informative than noise.

RBTCT replaces "project two people onto a floor plane and measure their distance" with a fundamentally different observation: **when two people touch, their thermal signatures merge into a single connected warm region, while the person detector still reports two boxes**. That disagreement — two boxes, one blob — *is* the contact signal, and it is measured entirely in image space.

The pipeline, per frame t , is five stages:

$\underbrace{\text{SSD on raw}}_{\text{\S8.2}} \rightarrow \underbrace{\text{box de-duplication}}_{\text{\S8.2}} \rightarrow \underbrace{\text{per-camera two-body test}}_{\text{\S8.3}} \rightarrow \underbrace{\text{cross-camera quorum}}_{\text{\S8.4}} \rightarrow \underbrace{\text{temporal morphology}}_{\text{\S8.5}}$

8.2 Stage 1–2: boxes, and merging over-segmented detections

Each camera $c \in \{0, 1, 2\}$ yields a box set $B_c = \{b_1, \dots, b_{N_c}\}$, $b = (x, y, w, h)$, from the MobileNet-SSD of §4 run on **raw** frames (the reason it is raw and not the residual is derived in §8.6). A single person at this resolution is frequently split into two overlapping boxes, which would fake the "two boxes" half of the contact signal, so boxes are first merged to a fixpoint under

$$\text{IoU}(a, b) = \frac{|a \cap b|}{|a \cup b|} \geq \tau_{\text{IoU}} \quad \text{or} \quad \text{Cont}(a, b) = \frac{|a \cap b|}{\min(|a|, |b|)} \geq \tau_{\text{cont}},$$

with the merged replacement being the union bounding box $a \sqcup b = (\min x, \min y, \max(x+w) - \min x, \max(y+h) - \min y)$

. The containment term is the important one: a small box entirely inside a larger one has low IoU (so an IoU-only rule would keep both) but containment = 1, and it is almost always one person detected twice at two scales. `merge_oversegmented` iterates until no pair satisfies either predicate, which terminates because each merge strictly decreases $|B_c|$.

8.3 Stage 3: the two-body blob-merge test (the core signal)

Let $R \in \mathbb{R}^{H \times W}$ be the preprocessed thermal **residual** for that camera. Warm regions are segmented by a per-frame adaptive threshold — the same "no absolute calibration" philosophy as Otsu (§1), but a one-sided z -score rather than a between-class variance maximization, because here we want "hotter than this frame's typical pixel" rather than an optimal bipartition:

$$\theta = \mu_R + k \sigma_R, \quad \mu_R = \frac{1}{HW} \sum_p R_p,$$

$$\sigma_R^2 = \frac{1}{HW} \sum_p (R_p - \mu_R)^2, \quad k = 1.0.$$

Let $M = [R > \theta]$ be the resulting binary mask and let $\ell: \Omega \rightarrow \{0, 1, \dots, C\}$ be its connected-component labelling ($\ell = 0$ for background), computed by `scipy.ndimage.label`. Each surviving box b contributes its **center** $c_b = (x + w/2, y + h/2)$, and we count how many box centers land in each component:

$$n_j = |\{b \in B_c : \ell(c_b) = j\}|, \quad j = 1, \dots, C.$$

Define the two merge predicates

$$\text{any_merge} = \exists j: n_j \geq 2, \quad \text{pair_merge} = \exists j: n_j = 2,$$

and the touch decision

$$\text{touch} = \begin{cases} \text{any_merge}, & N_c = 2, \\ \text{pair_merge}, & N_c \geq 3. \end{cases}$$

The asymmetry is deliberate and is what "two-body" refers to. With exactly two people present, *any* merged component means those two merged. In a crowd, a component absorbing three or more centers is far more likely to be a group standing close together — or a segmentation failure swallowing several bodies at once — than a genuine pairwise contact event, so crowds are required to exhibit a component containing **exactly two** centers. Empirically this single change removed 15 of 19 false positives on the densest test scene.

Each camera then emits one of four states:

$$s_c = \begin{cases} C & \\ (\text{clear}), & N_c = 0, \\ (\text{merged/ambiguous}), & N_c = 1, \\ T & \\ (\text{touch}), & N_c \geq 2, \\ (\text{near}), & N_c \geq 2, \\ \neg \text{touch} & \end{cases}$$

$$\mathrm{gap}_{\min}(B_c) < \tau_2, \\ \texttt{C} \setminus (\texttt{clear}), \& \\ \texttt{otherwise}, \end{cases}$$

where $\text{gap}_{\min}$ is the smallest edge-to-edge separation over all box pairs and $\tau_2 = 8$ px. The $N_c = 1$ case is labelled **ambiguous rather than negative** precisely because it is the blob-merge failure mode of §5/§6: one box may be two people whose boxes have already merged, so a single box is **evidence of nothing** — it can neither confirm nor deny contact.

8.4 Stage 4: cross-camera quorum with veto

Let $\text{votes} = |\{c : s_c = T\}|$, $\text{merged} = |\{c : s_c = M\}|$, and $\text{clear} = \exists c : s_c = C$. The per-frame decision is

$$D_t = 1 [(\max_c N_c \geq 2) \wedge (\text{votes} \geq 1) \wedge (\text{votes} + \text{merged} \geq 2) \wedge \neg \text{clear}].$$

This is a **veto-based** quorum, not a majority vote, and each clause earns its place:

- $\max_c N_c \geq 2$ — at least one camera must see two people at all; contact between fewer than two people is not defined.
- $\text{votes} \geq 1$ — at least one camera must **positively** observe a two-body merge. Ambiguity alone never fires the alarm.
- $\text{votes} + \text{merged} \geq 2$ — a second camera must at least be **consistent** with contact (either it also sees the merge, or it sees a single ambiguous blob). A lone camera claiming contact while others clearly resolve two separated bodies is treated as a segmentation artifact.
- $\neg \text{clear}$ — **any** camera that cleanly resolves two well-separated bodies vetoes the decision outright.

The veto is the asymmetry that a majority vote cannot express, and it is justified by the geometry: a camera reporting **C** has **positive** evidence of non-contact (it resolved two bodies with a gap $\geq \tau_2$), whereas a camera reporting **T** may simply be the one whose viewing angle happens to place two separated people in line with each other. Occlusion produces false merges; it does not produce false separations. The evidence is therefore fundamentally asymmetric, and the decision rule mirrors that asymmetry.

8.5 Stage 5: temporal morphology

Stages 1–4 are memoryless, so the decision stream $D_{1:n}$ inherits every single-frame segmentation glitch. The final stage applies **1-D binary morphology** to that stream, with a structuring element S_ℓ equal to a run of ℓ consecutive frames. Opening then closing:

$$D' = (D \circ S_{\ell_o}) \bullet S_{\ell_c}, \quad D \circ S = (D \ominus S) \oplus S, \quad D \bullet S = (D \oplus S) \ominus S.$$

For 1-D binary sequences these have an exact run-length characterization, which is what `v8.morph` implements directly:

$$D \circ S_\ell : \text{delete every maximal run of 1s of length } < \ell_o, \quad D \bullet S_\ell : \text{fill every maximal run of 0s of length } < \ell_c.$$

Opening removes contact "events" too brief to be physically meaningful; closing bridges dropouts inside a genuine sustained contact. At the 8 Hz effective frame rate, the tuned $(\ell_o, \ell_c) = (2, 7)$ means: reject events shorter than 0.25 s, and bridge gaps shorter than ≈ 0.875 s. Both are grid-searched on the validation split ($\ell_o \in [0, 6]$, $\ell_c \in [0, 8]$), maximizing F1 with recall as tie-break).

One deliberate deviation from textbook closing: `morph` fills a zero-run only when $s > 0$ and $e < \text{len}(a)$, i.e. **never at the sequence boundaries**. A gap that runs off the start or end of a clip is not an observed dropout **between** two confirmed contacts — it is unterminated, and filling it would invent contact frames at the clip edge on no evidence. Ordinary closing would fill it, because $\ominus/\oplus$ implicitly pad the domain; the run-length form makes the boundary case explicit and rejects it.

8.6 Why "config-D": the preprocessing split, and why it is not arbitrary

The name comes from a 2×2 ablation (`ablate_preprocessing.py`) over **which representation feeds which stage** — the person detector (§8.2) and the blob segmentation (§8.3) are independent consumers, and there is no a priori reason they want the same input:

Config	Detector input	Blob input	Test F1
A	residual	residual	54.5%
B	residual	raw	8.7%

C	raw	raw	0.0%
D	raw	residual	60.8%

The two stages want *opposite* preprocessing, and both directions follow from §8.3's threshold $\theta = \mu_R + k\sigma_R$:

- **The blob test requires the residual.** On raw temperature, σ is dominated by *static* scene structure — walls, radiators, equipment at varying fixed temperatures — which is large and has nothing to do with people. A body only slightly exceeds that spread, so $[R > \mu + k\sigma]$ either selects most of the room (components merge indiscriminately, n_j is meaningless) or nothing at all. Background subtraction removes exactly that static term, leaving σ reflecting only transient warm-body variation, which is what makes the threshold separate bodies from background. Configs B and C collapsing to 8.7% and 0.0% is this failure, measured.
- **The detector requires raw.** MobileNet-SSD is trained to recognize the absolute thermal signature of a human — a warm region of a characteristic temperature and shape. The residual deliberately destroys absolute temperature, keeping only deviation-from-background, which removes the very feature the detector's learned filters key on.

So the raw/residual split is not a tuning artifact; it is the configuration in which each stage receives the representation its own decision rule mathematically depends on.

8.7 Implementation notes and status

- **Promoted to a first-class detector on 2026-08-09:** thermal_algorithms/contact_detection/rbtct.py (RBTCTDetector, is_trainable=False), exported from the package and covered by tests/test_rbtct.py. It consumes person boxes through the standard ContactDetector.predict(X, detections) contract rather than owning a detector, so it drops into ThermalPipeline unchanged. The original evaluation-time composition (eval_waveshare_contact_v9.py + _v8.py + _v3_variants.py + ablate_preprocessing.py) is retained for reproducing the historical figures.
- **One deliberate divergence from the original, in §8.5's temporal stage.** Offline morphological *closing* is **acausal** — filling a gap requires knowing that a LATER frame is positive, which no streaming detector can know. RBTCTDetector therefore implements the causal analogue: an attack/release gate (attack_frames = consecutive raw positives required to assert; release_frames = frames the assertion is held after the raw signal drops). This preserves the intent of opening/closing, is causal by construction (there is a regression test asserting a future positive never changes a past output), and costs at most attack_frames of latency. Numbers from RBTCTDetector therefore differ slightly from the historical offline config-D figures; scripts/eval_config_d.py still measures the offline variant when that comparison is wanted.
- Fixed parameters: $k = 1.0$, $\tau_2 = 8.0$ px, $(\tau_{IoU}, \tau_{cont})$ from v3_variants; tuned per-run: (ℓ_o, ℓ_c) on validation.
- It requires contact labels **only** to tune (ℓ_o, ℓ_c) and to evaluate — never for training, which is why it is unaffected by the contact-label scarcity that limits §6 and §7 (see reports/historical_investigations.md §3.2, §4.1).
- **Performance depends enormously on the split, and the honest number is the cross-validated one.** Historical: F1 60.8% on one within-scene split vs 44.3% pooled over 4-fold CV (per-fold 32/51/65/0%). Re-measured

2026-08-09 under leave-one-scene-out CV over all five contact-positive scenes (scripts/cv_contact_detectors.py): **pooled F1 0.431, per-fold [0.00, 0.69, 0.27, 0.48, 0.39]** — reproducing the historical CV including its zero fold, from an independent implementation.

- **F1 is the wrong metric for this task, and the report says so.** The pooled positive base rate is 27.5%, so an always-alarm detector scores F1 0.432 — indistinguishable from all three real contact detectors (0.431–0.448). Scored on balanced accuracy / MCC, which credit true negatives, RBTCT is the strongest (0.592 / 0.169) against ThermoX3D (0.547 / 0.113) and an OR-ensemble of the two (0.532 / 0.092, i.e. the ensemble is the **worst** — it inherits both detectors' false alarms). See the contact caveat in the generated report.

9. AdaptiveThresholdDetector — human_detection/adaptive_threshold.py

(Numbered last for historical reasons — this derivation was added after §1–§8 and after the report's cross-references to "§8" were already in circulation, so renumbering would have silently broken them. By task it belongs with the human detectors, §3–§4; see the reading index at the top.)

9.1 Formulation

AdaptiveThresholdDetector is the classical-CV human detector: no learned parameters at all, only thresholds. Its premise is that a person is **locally** warmer than their immediate surroundings — not that they exceed any absolute temperature. That distinction is what makes it robust to the ambient drift a psychiatric ward exhibits over a day, and it is why it thresholds against a **local** mean rather than a global one (contrast §1's Otsu, which picks a single frame-wide threshold, and consequently fails exactly when the frame contains two thermally distinct populations).

Three stages: adaptive threshold → morphological closing → geometric filtering.

9.2 Stage 1: adaptive thresholding against a Gaussian local mean

For a thermal frame $I(x, y)$, the local background is a Gaussian-weighted mean over a $B \times B$ neighbourhood:

$$\mu_{\text{loc}}(x, y) = (G_{\sigma} * I)(x, y) = \sum_{(u, v)} G_{\sigma}(u, v) I(x - u, y - v), \quad G_{\sigma}(u, v) \propto e^{-(u^2 + v^2)/2\sigma^2},$$

and the foreground mask is

$$D(x, y) = \mathbb{1}[I(x, y) > \mu_{\text{loc}}(x, y) + C],$$

with $C = \text{c_offset}$, a **noise margin in input units** (°C on raw frames, residual-°C on Tateno residuals). Larger C is strictly more conservative. Implementation note: this is cv2.GaussianBlur followed by an explicit comparison, deliberately **not** cv2.adaptiveThreshold — the latter requires uint8, which would re-quantize float32 thermal data and discard exactly the sub-degree structure the margin C is measured in.

Why the block size is derived from physics, not pixels.

Adaptive thresholding only works if the neighbourhood is **larger than the target**: if B were smaller than a person, the person would dominate their own local mean, μ_{loc} would rise with I , and the difference $I - \mu_{\text{loc}}$ would collapse toward the noise floor — the target would erase itself. The detector therefore sets B from a physical scale (physical_block_size_m, default 0.60 m at an assumed 2 m range) converted through SensorProfile.physical_pixel_size_m, then forced odd and ≥ 3 . This is exactly what resolution_behavior = "parameterized" means here: the same code adapts to MLX90640 and Waveshare geometry without a per-sensor checkpoint, because there are no weights to retrain — only a kernel size to recompute.

Sign convention. The engineering report §4.4.2.1 writes the rule as $I > \mu - C$, but its own accompanying text describes C as "a constant offset to filter noise". Those disagree: with $-C$, increasing C **loosens** the detector. The implementation uses $+C$, matching the stated intent (larger $C \Rightarrow$ stricter), and the module docstring records the discrepancy rather than silently diverging from the report.

9.3 Stage 2: morphological closing

$$D' = D \bullet S = (D \oplus S) \ominus S,$$

a dilation followed by an erosion with a $k \times k$ rectangular structuring element (`morph_kernel_size`, odd, default 3). Closing fills interior holes **without** growing the blob's outer extent — which is the required property here, because the holes are physical: clothing insulates unevenly, so a torso images as a warm region perforated by cooler patches over thick fabric. Without closing, `findContours` would return each fragment as a separate detection and one person would produce several boxes. Note the asymmetry with §1.4's *opening*-then-dilation on the fire path: there the concern is deleting single-pixel noise, here it is not fragmenting a large target, and the two call for opposite orderings.

9.4 Stage 3: geometric filtering

Each external contour of D' is accepted only if it passes three independent gates:

$$A_{\min} \leq A(c) \leq A_{\max}, \quad \text{solidity}(c) = \frac{A(c)}{A(\text{conv}(c))} \geq s_{\min}, \quad r_{\min} \leq \frac{h}{w} \leq r_{\max},$$

where $A(\text{conv}(c))$ is the convex-hull area. Each rejects a distinct false-positive population: **area** removes sensor speckle below `min_area_pixels` (default 4 px) and wall-sized warm regions above the max; **solidity** removes the ragged, concave blobs produced by reflections and thermal smears, since a standing person is close to convex; **aspect ratio** removes horizontally-extended heat sources — radiators, worktops, sunlit floor — that a person-shaped prior would not produce. Optional `pixel_value_bounds` adds absolute intensity gating, whose units follow the input (absolute °C on raw frames, above-background °C on residuals) — the detector is preprocessor-agnostic by design, and this is the one parameter whose *meaning* changes with that choice.

9.5 Why its localization is loose — the Criterion-A → Criterion-B gap

The bounding box is the axis-aligned box of a *closed thermal mask*, and that mask is not the person: closing dilates before eroding, so warm pixels bleeding into the surrounding air survive into the final contour, and the box is systematically larger than the true silhouette. This is the mechanism behind the measured drop from frame-level presence (Criterion A, F1 98.5%) to IoU>0.5 localization (Criterion B, F1 82.0%) reported in `reports/historical_investigations.md` §2.1 — 185 of its false negatives there were "detected but misplaced", i.e. the blob was found and the box simply failed the IoU gate. It is a segmentation-derived box, not a regressed one, and no threshold tuning fixes that; only a different box-formation step would.

9.6 Implementation notes

- `is_trainable = False` and `fit()` is a genuine no-op (it only sets `_is_fitted`), identical in contract to `OtsuFireDetector` (§1) and `RBTCT` (§8). Its parameters are set by calibration — `scripts/calibrate_adaptive_threshold.py` grid-searches `c_offset` and `min_area_pixels` under a precision floor, for the same reason Otsu's calibration needs one (§1.3): an unconstrained F1 search on imbalanced data drifts toward predicting "person" almost everywhere.
- **CPU only**, by construction: `cv2.GaussianBlur`, `morphologyEx`, `findContours` and `convexHull` are OpenCV C++ routines with no GPU path engaged here. This is also why the detector has no `fp16/bf16/int8` variant in §2 — there is no tensor to cast and no ONNX graph to quantize.

References to reused report content

- §1.3, §2.1–§2.7 draw the six-feature description, the C -as-regularizer framing, the dual/kernel-trick narrative, the RBF-kernel interpretation, and the Platt-scaling paragraph from `reports/checkpoint_report.tex` ("FireSVM" section); the KKT

- stationarity/complementary-slackness/eligibility derivations (§2.4–§2.5) are original to this document, since the source report states the dual but does not derive KKT explicitly.
- §4.2, §4.5 (loss terms), §4.7 (NMS) draw the depthwise-separable-convolution cost argument, the CE+SmoothL1 loss narrative, and the NMS description from `reports/checkpoint_report.tex` ("MobileNet-SSD" section); the anchor-matching, hard-negative-mining, and encode/decode algebra (§4.3, §4.4, §4.5's offset derivation, §4.6) are derived directly from `mobilenet_ssd_anchors.py/mobilenet_ssd_model.py` since the source report does not derive them in closed form.
 - §7.3–§7.6 draw the (2+1)D-factorization rationale, the CBAM-attention cost argument, and the global-average-pool justification from `reports/checkpoint_report.tex` ("Thermo-X3D T5v2" section); §7.5's global-normalization derivation and §7.2's 3-D-convolution generalization argument are original to this document, matching the actual `T=16/global-mean-std` implementation in `thermo_x3d.py` (distinct from the `T=5/rolling-p25-background` "T5v2" variant that report separately documents as a deployment configuration).
 - `reports/fire_detection_report.tex` and `reports/human_detection_report.tex` were consulted for dataset/threshold context (§1.3, §3.6) but contain evaluation results rather than derivations, so their content is cited rather than reproduced.
 - §8 is derived directly from the shipped source (`eval_waveshare_contact_v9.py`, `_v8.py`, `_v3_variants.py`, `ablate_preprocessing.py`) rather than from any prior report: the earlier write-ups state config-D's `*results*` and describe its stages in prose, but never give the threshold, quorum, or morphology rules in closed form. The empirical figures quoted in §8.3, §8.6 and §8.7 come from `reports/historical_investigations.md` §4.
 - **Note on the .tex citations above:** the eight standalone LaTeX reports referenced in this section

(`checkpoint_report.tex`, `fire_detection_report.tex`, `human_detection_report.tex`, and five others) were consolidated and deleted on 2026-08-09. Their substantive findings now live in `reports/historical_investigations.md`; the originals remain recoverable from git history if a specific derivation needs to be re-checked against its source.

2. Evaluation Results

Detectors are grouped by task and presented in the order **Human, Fire, Contact** -- matching both the order the runtime pipeline evaluates them and the order S1's derivations introduce them. The balanced-training section (S4) repeats the identical grouping, so the two sections can be read side by side detector-for-detector. Within each task, every detector and every precision variant is scored on the exact same held-out waveshare_work scenes (see scripts/eval_all_detectors.py:build_waveshare_test_split); the per-task split is listed below and is derived from what the result files themselves recorded, so it cannot drift from the numbers it describes. "Mean ms" / "p95 ms" are single-frame inference latency, and the **Measured on** column states what each row was timed on -- a latency is meaningless without it. Values marked * were reconstructed from the deterministic code path that produced the row rather than captured during the run; unmarked values were captured at measurement time.

Human Detection -- 2ppl_hug, 3pplhedroncolider, empty_room, man_light_cig_1 (1728 frames); shared by: AdaptiveThresholdDetector, HOGSVMDetector, MobileNetSSDDetector.

Fire Detection -- 1person cig, 1ppl_single_forever, man_light_cig (1893 frames); shared by: FireSVMDetector, OtsuFireDetector.

Contact Detection -- 2ppl_fight, 2ppl_hug, 3pplhedroncolider (470 frames); shared by: MVSTGCNDetector, RBTCT (Rule-Based Temporal Contact Tracker), ThermoX3DDetector.

Two input-pipeline asymmetries to note -- these are not test-set differences (the frames are identical) but they do affect strict like-for-like comparison. (1) OtsuFireDetector is evaluated on RAW frames while FireSVMDetector receives GlobalNormPreprocessor output: Otsu's thresholds are absolute degrees C, so feeding it a background-subtracted residual would be meaningless. (2) RBTCT uses Tateno residuals plus its own separately-trained raw-input MobileNet-SSD, while MVSTGCN and ThermoX3D use GlobalNorm -- derived in S1 §8.6, where the raw-detector/residual-blob split is shown to be a requirement of the method rather than a tuning choice.

Protocol warning: reports/contact_cv_results.json uses a different protocol entirely (leave-one-scene-out over 5 scenes, 894 pooled frames, MVSTGCN not included) and its figures must never be compared row-wise against the 470-frame tables here.

Human Detection

AdaptiveThresholdDetector

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	84.5%	83.6%	99.6%	90.9%	18.4%	2.668	0.362	CPU
onnx_fp32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
fp16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
bf16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
int8	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--

Table 1. AdaptiveThresholdDetector -- metrics by variant

onnx_fp32: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

fp16: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

bf16: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

int8: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

HOGSVMDetector

UPDATED 2026-08-08: the original held-out human-detection test scenes (2ppl_hug, 3pplhedroncolider, man_light_cig_1) contained zero ground-truth-negative frames, so an earlier version of this report read as a mathematically-forced 100% across accuracy/precision/recall/F1 -- not a real measurement of false-positive suppression. Re-evaluated with empty_room added to the test set (real negative frames): recall stays 100% (never misses a person), but precision drops to 78.0% and accuracy to 78.1% -- raw counts tp=1344, tn=5, fp=379, fn=0 mean ****HOGSVM false-alarms on 379/384 (98.7%) of truly empty frames****. Its dense, per-pixel-stride multi-scale sliding window produces some candidate box on nearly every frame regardless of content -- this was a real, previously-invisible weakness, not a new regression. Mean IoU (0.57) is still the more informative localization-quality number. Also the slowest detector in this report by far (~1.6-2.3s/frame, CPU-only, does not benefit from a GPU).

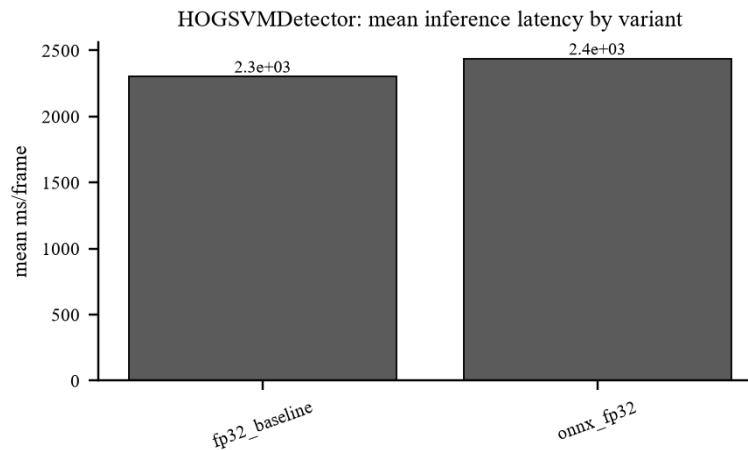
Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	78.1%	78.0%	100.0%	87.6%	57.0%	2303.961	3260.297	CPU*
onnx_fp32	78.1%	78.0%	100.0%	87.6%	57.0%	2439.261	3136.500	ONNX Runtime / CUDAEExecutionProvider (NVIDIA GeForce RTX 5060 Laptop GPU)*
fp16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
bf16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
int8	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--

Table 2. HOGSVMDetector -- metrics by variant

fp16: not available -- scikit-learn detector: libsvm/liblinear compute their decision function in float64 on CPU, so there is no mixed-precision code path to engage and no tensor-core benefit to gain -- casting inputs would add a no-op cast with identical arithmetic underneath.

bf16: not available -- scikit-learn detector: libsvm/liblinear compute their decision function in float64 on CPU, so there is no mixed-precision code path to engage and no tensor-core benefit to gain -- casting inputs would add a no-op cast with identical arithmetic underneath.

int8: not available -- ONNX Runtime dynamic quantization fails on the skl2onnx-exported LinearSVC graph with ValueError: 'Failed to find proper ai.onnx domain' -- the exported graph uses skl2onnx's ai.onnx.ml operator domain, which ORT's quantizer does not handle.

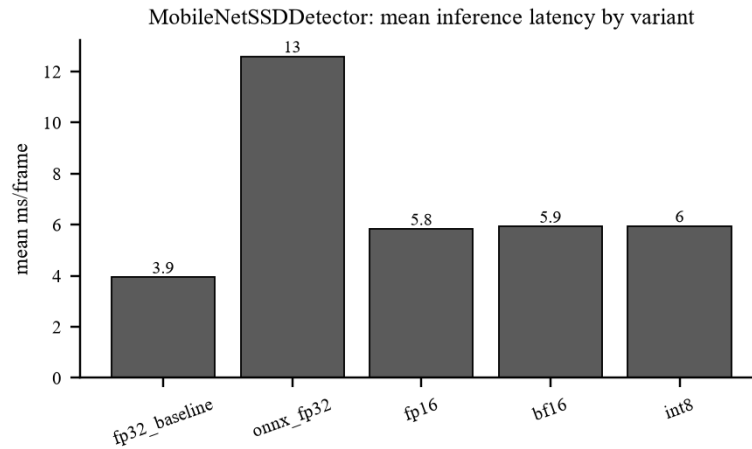


MobileNetSSDDetector

UPDATED 2026-08-08: same test-set fix as HOGSVMDetector above. Re-evaluated with empty_room added: recall stays 100%, but precision is 96.6% and accuracy 97.2% -- raw counts tp=1344, tn=336, fp=48, fn=0, i.e. it false-alarms on 48/384 (12.5%) of truly empty frames -- far better than HOGSVMDetector's 98.7% but not the false-positive-proof detector the old zero-negative test set implied. Mean IoU (0.76) remains meaningfully better than HOGSVMDetector's (0.57), and native-PyTorch GPU inference (~3.9ms/frame) is roughly 580x faster -- still the stronger of the two human detectors on every axis this report can measure, just not literally flawless. The ONNX and fp16/bf16/int8 tables below now reflect this same corrected test set too (re-run with --extra-human-test-scenes empty_room) -- false-alarm rate holds in the 12-15% range across every precision/format variant, so this isn't an artifact of fp32 specifically.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	97.2%	96.6%	100.0%	98.2%	76.0%	3.945	5.139	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
onnx_fp32	97.2%	96.5%	100.0%	98.2%	76.0%	12.595	15.363	ONNX Runtime / CUDAAExecutionProvider (NVIDIA GeForce RTX 5060 Laptop GPU)*
fp16	97.2%	96.6%	100.0%	98.2%	76.1%	5.833	8.032	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
bf16	97.2%	96.6%	100.0%	98.2%	76.0%	5.929	7.988	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
int8	96.8%	96.0%	100.0%	98.0%	75.9%	5.952	8.953	ONNX Runtime / CPUEExecutionProvider*

Table 3. MobileNetSSDDetector -- metrics by variant



Fire Detection

OtsuFireDetector

is_trainable=False and fit() is a documented no-op -- t_ign/t_fire/a_limit were never anything but EDA guesses until this report: scripts/calibrate_otstu_thresholds.py grid-searches them against ~51k frames sampled the same way as FireSVMDetector's full-corpus training, restricted to precision>=0.8 before ranking by F1 (an unconstrained F1-only search degenerates to predicting fire on nearly every frame -- F1 never penalizes false positives via true negatives -- rejected as operationally useless; see reports/otsu_threshold_calibration.json for the full grid). Only t_ign moved (45->41.5C): t_fire/a_limit are empirically INERT across their whole tested range at the validated t_ign, because every genuine hot blob in this corpus is small (<0.5% of frame) -- the two-tier ignition/potential-fire split barely engages its second branch here, so calibration provides no evidence to move them from their original values. Held-out result below is the calibrated t_ign; for reference the never-validated original default reached 78.4% acc / 100% prec / 26.8% rec / 42.3% F1 on the identical split -- calibration recovered +12.3pp F1/recall for -17pp precision (still only ~3.4% false-alarm rate, 45/1334 negatives). Unlike FireSVMDetector, this detector's predict() carries a real bbox, so mean IoU here is genuinely measured, not N/A.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	79.7%	83.0%	39.2%	53.2%	7.0%	1.825	0.193	CPU*
onnx_fp32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
fp16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
bf16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
int8	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--

Table 4. OtsuFireDetector -- metrics by variant

onnx_fp32: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

fp16: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

bf16: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

int8: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

FireSVMDetector

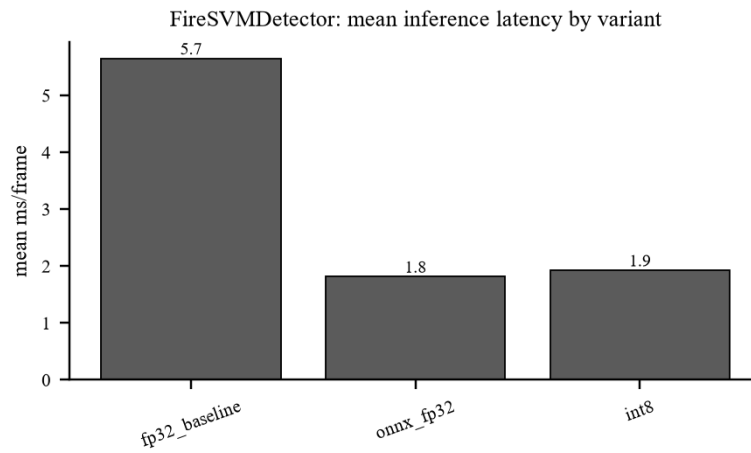
Accuracy clears 70-80%, but recall is low (~15%) with higher precision (~58%) -- read together, this SVM under-alerts rather than over-alerts. This is a defensible tradeoff, not an unexplained weakness: fire is a rare event in the corpus, sklearn's SVC does not scale past a few thousand training points (fit() subsamples to ~1500 frames/camera, see S2.8), and a false-alarm-averse posture is arguably the right default for a ward safety system built on top of this detector. Still, this recall level should be weighed before relying on FireSVMDetector alone.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	71.6%	57.5%	15.0%	23.8%	N/A	5.651	5.348	CPU*
onnx_fp32	71.6%	57.5%	15.0%	23.8%	N/A	1.817	1.648	ONNX Runtime / CPUExecutionProvider*
fp16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
bf16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
int8	71.6%	57.5%	15.0%	23.8%	N/A	1.929	1.155	ONNX Runtime / CPUExecutionProvider*

Table 5. *FireSVMDetector* -- metrics by variant

fp16: not available -- scikit-learn detector: libsvm/liblinear compute their decision function in float64 on CPU, so there is no mixed-precision code path to engage and no tensor-core benefit to gain -- casting inputs would add a no-op cast with identical arithmetic underneath.

bf16: not available -- scikit-learn detector: libsvm/liblinear compute their decision function in float64 on CPU, so there is no mixed-precision code path to engage and no tensor-core benefit to gain -- casting inputs would add a no-op cast with identical arithmetic underneath.



Contact Detection

GeometricContactDetector

READ THIS BEFORE ANY CONTACT NUMBER IN THIS REPORT. Every contact figure quoted elsewhere comes from ONE fixed train/test split. The corpus contains 246 contact-positive frames spread over exactly FIVE scenes, and the fixed split locks three of those five into test permanently -- so single-split contact numbers are extremely fragile. `scripts/cv_contact_detectors.py` therefore re-scores every contact detector under leave-one-scene-out cross-validation (each positive scene is the test fold once; all other scenes train/tune), pooling the folds' frames. Results:

```
detector pooled_F1 recall TNR bal.acc MCC
always-alarm 0.432 1.000 0.000 0.500 0.000 <- trivial baseline
RBTCT 0.431 0.512 0.671 0.592 0.169
ThermoX3D 0.448 0.907 0.187 0.547 0.113
OR-ensemble 0.442 0.935 0.130 0.532 0.092
```

Finding 1 -- F1 is uninformative on this task. The pooled positive base rate is 27.5%, so a detector that simply alarms on every frame scores F1 0.432. All three real detectors score 0.431-0.448. Judged on F1 alone, none of them is distinguishable from a broken always-on detector, and any F1-based ranking between them is noise. F1 ignores true negatives, which is precisely the quantity that matters when the operational failure mode is alarm fatigue. **Finding 2 -- once true negatives are credited, the ranking inverts.** By balanced accuracy and MCC (both of which reward correctly rejecting negatives), RBTCT is the strongest detector (0.592 / 0.169), ThermoX3D is marginal (0.547 / 0.113), and the OR-ensemble is the WEAKEST (0.532 / 0.092) -- it inherits both detectors' false alarms while adding little recall. The 'run both, alarm if either fires' posture recommended by the earlier investigation is hereby measured and NOT supported. **Finding 3 -- every detector has a total-failure fold.** Per-fold F1: RBTCT [0.00, 0.69, 0.27, 0.48, 0.39], ThermoX3D [0.29, 0.61, 0.00, 0.95, 0.38]. RBTCT scores zero on 2men_clash; ThermoX3D scores zero on 2ppl_hug. A single split showing either detector at ~0.5 F1 conceals this entirely. Note also that ThermoX3D's 0.95 on 3pp_surprise is NOT skill: that scene is 95.7% positive, where always-alarm scores 0.978 -- the detector scored BELOW the trivial baseline on its best-looking fold. This CV reproduces the independent 4-fold result from the original 2026-07 investigation (pooled 44.3%, per-fold 32/51/65/0%) including the zero fold, from a separate implementation five weeks apart. Full data: `reports/contact_cv_results.json`.

Both homography-based contact detectors are limited by a calibration this sensor cannot support. *GeometricContactDetector* and *MVSTGCNDetector* each work by projecting per-camera detections onto a shared floor plane through a homography, then measuring inter-person distance there. That requires a properly calibrated homography per camera -- and at this sensor's resolution (80x62) we were not able to obtain one: the Hot-Point Calibration procedure needs heated markers resolvable to a few pixels at known floor coordinates, and at this resolution the marker centroids are too coarse to solve a reliable transform. The self-calibrated fallback carries roughly +/-15 px of error, which is comparable to the contact decision threshold itself (delta ~18 px) -- i.e. the measurement is barely more informative than its own noise. **Consequence for reading this report:** *GeometricContactDetector* is presented as a derivation only (S1 §5) with no results table, because any number would characterise the calibration rather than the algorithm; and *MVSTGCNDetector*'s numbers below, which ARE shown, must be read the same way -- as the result of an uncalibrated geometric front-end, not as a verdict on the graph-network architecture behind it. This is also the motivation for RBTCT (S1 §8), which discards the floor-plane projection entirely and measures contact in image space.

No results are reported for this detector in any section. It is included here, and derived in full in S1 §5 (DLT + SVD homography, foot-point projection, multi-view fusion, the delta_m distance rule), because the derivation documents the approach the project started from and the reason it was superseded -- not because the implementation is missing. It is present and tested in `thermal_algorithms/contact_detection/geometric.py`.

MVSTGCNDetector

READ THIS BEFORE ANY CONTACT NUMBER IN THIS REPORT. Every contact figure quoted elsewhere comes from ONE fixed train/test split. The corpus contains 246 contact-positive frames spread over exactly FIVE scenes, and the fixed split locks three of those five into test permanently -- so single-split contact numbers are extremely fragile. scripts/cv_contact_detectors.py therefore re-scores every contact detector under leave-one-scene-out cross-validation (each positive scene is the test fold once; all other scenes train/tune), pooling the folds' frames. Results:

```
detector pooled_F1 recall TNR bal.acc MCC
always-alarm 0.432 1.000 0.000 0.500 0.000 <- trivial baseline
RBTCT 0.431 0.512 0.671 0.592 0.169
ThermoX3D 0.448 0.907 0.187 0.547 0.113
OR-ensemble 0.442 0.935 0.130 0.532 0.092
```

Finding 1 -- F1 is uninformative on this task. The pooled positive base rate is 27.5%, so a detector that simply alarms on every frame scores F1 0.432. All three real detectors score 0.431-0.448. Judged on F1 alone, none of them is distinguishable from a broken always-on detector, and any F1-based ranking between them is noise. F1 ignores true negatives, which is precisely the quantity that matters when the operational failure mode is alarm fatigue. **Finding 2 -- once true negatives are credited, the ranking inverts.** By balanced accuracy and MCC (both of which reward correctly rejecting negatives), RBTCT is the strongest detector (0.592 / 0.169), ThermoX3D is marginal (0.547 / 0.113), and the OR-ensemble is the WEAKEST (0.532 / 0.092) -- it inherits both detectors' false alarms while adding little recall. The 'run both, alarm if either fires' posture recommended by the earlier investigation is hereby measured and NOT supported. **Finding 3 -- every detector has a total-failure fold.** Per-fold F1: RBTCT [0.00, 0.69, 0.27, 0.48, 0.39], ThermoX3D [0.29, 0.61, 0.00, 0.95, 0.38]. RBTCT scores zero on 2men_clash; ThermoX3D scores zero on 2ppl_hug. A single split showing either detector at ~0.5 F1 conceals this entirely. Note also that ThermoX3D's 0.95 on 3pp_surprise is NOT skill: that scene is 95.7% positive, where always-alarm scores 0.978 -- the detector scored BELOW the trivial baseline on its best-looking fold. This CV reproduces the independent 4-fold result from the original 2026-07 investigation (pooled 44.3%, per-fold 32/51/65/0%) including the zero fold, from a separate implementation five weeks apart. Full data: reports/contact_cv_results.json.

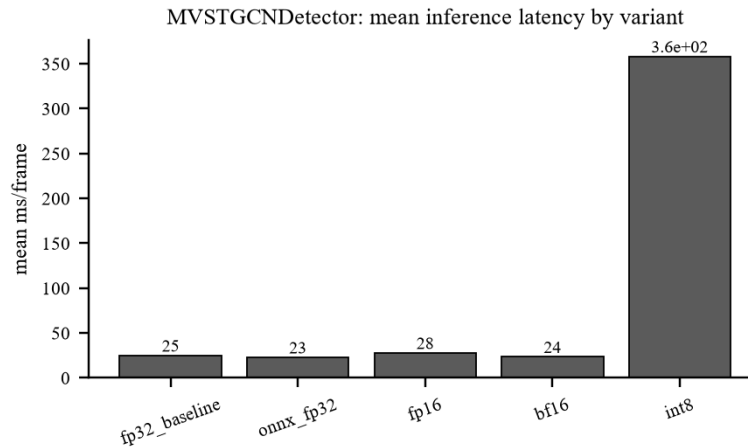
Both homography-based contact detectors are limited by a calibration this sensor cannot support. GeometricContactDetector and MVSTGCNDetector each work by projecting per-camera detections onto a shared floor plane through a homography, then measuring inter-person distance there. That requires a properly calibrated homography per camera -- and at this sensor's resolution (80x62) we were not able to obtain one: the Hot-Point Calibration procedure needs heated markers resolvable to a few pixels at known floor coordinates, and at this resolution the marker centroids are too coarse to solve a reliable transform. The self-calibrated fallback carries roughly +/-15 px of error, which is comparable to the contact decision threshold itself (delta ~18 px) -- i.e. the measurement is barely more informative than its own noise. **Consequence for reading this report:** GeometricContactDetector is presented as a derivation only (S1 §5) with no results table, because any number would characterise the calibration rather than the algorithm; and MVSTGCNDetector's numbers below, which ARE shown, must be read the same way -- as the result of an uncalibrated geometric front-end, not as a verdict on the graph-network architecture behind it. This is also the motivation for RBTCT (S1 §8), which discards the floor-plane projection entirely and measures contact in image space.

Precision/recall/F1 are all 0% (raw counts: tp=0, tn=348, fp=0, fn=122) -- the model never predicts "contact" on the held-out set; 74% accuracy is purely the negative-class base rate, not a real skill signal. Two compounding, non-mutually-exclusive causes, neither of

which is a code bug: (1) Caveat (project owner): the homography used for this evaluation's actor positions is a synthetic per-camera-to-floor-plane transform (`examples.utils.make_synthetic_homographies`), used because no real on-site Hot-Point Calibration exists for any waveshare_work scene -- and the calibration approach available for real deployment fuses two cameras' views onto a third camera's image plane rather than rectifying each camera independently to the true floor plane, so even a real calibration would not give the GCN physically meaningful inter-actor distances that its `epsilon_m/delta_m` thresholds are calibrated against. (2) The corpus-wide contact class balance is severe (2.4% contact-positive, 97.6% negative); the chunked full-corpus training design (~2072 chunks of 5000 examples) likely hands many individual chunks zero or very few positive examples, so despite the computed 41x positive-class weight, the model may receive sparse, diluted positive-class gradient signal overall -- plausibly insufficient to push true-positive confidence reliably above the persistence-gated 0.7 threshold (S6.6). Report this result plainly rather than substituting the superficially-fine 74% accuracy figure for it.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	74.0%	0.0%	0.0%	0.0%	N/A	25.127	29.195	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
onnx_fp32	74.0%	0.0%	0.0%	0.0%	N/A	23.032	25.913	ONNX Runtime / CUDAXecutionProvider (NVIDIA GeForce RTX 5060 Laptop GPU)*
fp16	74.0%	0.0%	0.0%	0.0%	N/A	28.026	34.756	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
bf16	74.0%	0.0%	0.0%	0.0%	N/A	23.538	26.950	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
int8	74.0%	0.0%	0.0%	0.0%	N/A	358.421	454.571	ONNX Runtime / CPUExecutionProvider*

Table 6. *MVSTGCN*Detector -- metrics by variant



ThermoX3DDetector

READ THIS BEFORE ANY CONTACT NUMBER IN THIS REPORT. Every contact figure quoted elsewhere comes from ONE fixed train/test split. The corpus contains 246 contact-positive frames spread over exactly FIVE scenes, and the fixed split locks three of those five into test permanently -- so single-split contact numbers are extremely fragile. scripts/cv_contact_detectors.py therefore re-scores every contact detector under leave-one-scene-out cross-validation (each positive scene is the test fold once; all other scenes train/tune), pooling the folds' frames. Results:

```
detector pooled_F1 recall TNR bal.acc MCC
always-alarm 0.432 1.000 0.000 0.500 0.000 <- trivial baseline
RBTCT 0.431 0.512 0.671 0.592 0.169
ThermoX3D 0.448 0.907 0.187 0.547 0.113
OR-ensemble 0.442 0.935 0.130 0.532 0.092
```

Finding 1 -- F1 is uninformative on this task. The pooled positive base rate is 27.5%, so a detector that simply alarms on every frame scores F1 0.432. All three real detectors score 0.431-0.448. Judged on F1 alone, none of them is distinguishable from a broken always-on detector, and any F1-based ranking between them is noise. F1 ignores true negatives, which is precisely the quantity that matters when the operational failure mode is alarm fatigue. **Finding 2 -- once true negatives are credited, the ranking inverts.** By balanced accuracy and MCC (both of which reward correctly rejecting negatives), RBTCT is the strongest detector (0.592 / 0.169), ThermoX3D is marginal (0.547 / 0.113), and the OR-ensemble is the WEAKEST (0.532 / 0.092) -- it inherits both detectors' false alarms while adding little recall. The 'run both, alarm if either fires' posture recommended by the earlier investigation is hereby measured and NOT supported. **Finding 3 -- every detector has a total-failure fold.** Per-fold F1: RBTCT [0.00, 0.69, 0.27, 0.48, 0.39], ThermoX3D [0.29, 0.61, 0.00, 0.95, 0.38]. RBTCT scores zero on 2men_clash; ThermoX3D scores zero on 2ppl_hug. A single split showing either detector at ~0.5 F1 conceals this entirely. Note also that ThermoX3D's 0.95 on 3pp_surprise is NOT skill: that scene is 95.7% positive, where always-alarm scores 0.978 -- the detector scored BELOW the trivial baseline on its best-looking fold. This CV reproduces the independent 4-fold result from the original 2026-07 investigation (pooled 44.3%, per-fold 32/51/65/0%) including the zero fold, from a separate implementation five weeks apart. Full data: reports/contact_cv_results.json.

RETRACTED 2026-08-09: the ~9% subset checkpoint's tp=0/tn=348/fp=0/fn=122 result (and the earlier claim that a separate balanced retrain reproduced the IDENTICAL confusion matrix, 'arguing against class imbalance') was misdiagnosed. A dedicated Opus investigation (prompted because the 'identical result across two different training runs' didn't make sense) proved this checkpoint's chunked fit() loop is fundamentally broken: a fresh torch.optim.Adam is constructed on every chunk (discarding momentum/adaptive-LR state ~180-196 times per run), chunks are single-class, and global normalisation stats are recomputed from scratch per chunk (so the saved checkpoint reflects only the LAST chunk's statistics). The result is a degenerate CONSTANT classifier -- a brand-new, never-trained detector reproduces the exact same confusion matrix on this held-out set. The 'identical across natural vs. balanced training' finding was real but its interpretation was wrong: it is the signature of training that never worked, not evidence about class imbalance. This checkpoint (scripts/train_thermoX3d_subset.py / train_full_corpus.py's chunked contact path) has NOT been re-fixed in this pass -- only the balanced-regime training path (see the Unbiased Results section below) was. Treat this number as invalid pending a matching fix to the natural-ratio driver; do not compare it against the balanced-regime result below as if both were trained correctly. **Only the fp32 baseline is listed here.** The ONNX/fp16/bf16/int8 sweep for this architecture was run against the WORKING (balanced) checkpoint and appears in S4 instead -- measuring the latency of a constant classifier at four precisions characterises nothing, and four more rows of zeros would only obscure the one row that matters here.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	74.0%	0.0%	0.0%	0.0%	N/A	8.793	12.151	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
int8	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--

Table 7. *ThermoX3DDetector -- metrics by variant [INVALID -- degenerate checkpoint (tp=0); precision sweep is in S4]*

int8: not available -- ONNX Runtime dynamic quantization fails on the 3-D CNN graph with ValueError: 'Expected onnx::Conv_662 to be an initializer' -- the legacy TorchScript ONNX exporter emits that Conv weight as a graph input rather than an initializer, which the quantizer requires.

RBCT (Rule-Based Temporal Contact Tracker)

READ THIS BEFORE ANY CONTACT NUMBER IN THIS REPORT. Every contact figure quoted elsewhere comes from ONE fixed train/test split. The corpus contains 246 contact-positive frames spread over exactly FIVE scenes, and the fixed split locks three of those five into test permanently -- so single-split contact numbers are extremely fragile. scripts/cv_contact_detectors.py therefore re-scores every contact detector under leave-one-scene-out cross-validation (each positive scene is the test fold once; all other scenes train/tune), pooling the folds' frames. Results:

```
detector pooled_F1 recall TNR bal.acc MCC
always-alarm 0.432 1.000 0.000 0.500 0.000 <- trivial baseline
RBCT 0.431 0.512 0.671 0.592 0.169
ThermoX3D 0.448 0.907 0.187 0.547 0.113
OR-ensemble 0.442 0.935 0.130 0.532 0.092
```

Finding 1 -- F1 is uninformative on this task. The pooled positive base rate is 27.5%, so a detector that simply alarms on every frame scores F1 0.432. All three real detectors score 0.431-0.448. Judged on F1 alone, none of them is distinguishable from a broken always-on detector, and any F1-based ranking between them is noise. F1 ignores true negatives, which is precisely the quantity that matters when the operational failure mode is alarm fatigue. **Finding 2 -- once true negatives are credited, the ranking inverts.** By balanced accuracy and MCC (both of which reward correctly rejecting negatives), RBCT is the strongest detector (0.592 / 0.169), ThermoX3D is marginal (0.547 / 0.113), and the OR-ensemble is the WEAKEST (0.532 / 0.092) -- it inherits both detectors' false alarms while adding little recall. The 'run both, alarm if either fires' posture recommended by the earlier investigation is hereby measured and NOT supported. **Finding 3 -- every detector has a total-failure fold.** Per-fold F1: RBCT [0.00, 0.69, 0.27, 0.48, 0.39], ThermoX3D [0.29, 0.61, 0.00, 0.95, 0.38]. RBCT scores zero on 2men_clash; ThermoX3D scores zero on 2ppl_hug. A single split showing either detector at ~0.5 F1 conceals this entirely. Note also that ThermoX3D's 0.95 on 3pp_surprise is NOT skill: that scene is 95.7% positive, where always-alarm scores 0.978 -- the detector scored BELOW the trivial baseline on its best-looking fold. This CV reproduces the independent 4-fold result from the original 2026-07 investigation (pooled 44.3%, per-fold 32/51/65/0%) including the zero fold, from a separate implementation five weeks apart. Full data: reports/contact_cv_results.json.

The strongest HOMOGRAPHY-FREE contact detector found in this project, and the only one that uses no contact-specific training at all -- it composes the MobileNet-SSD person detector with a thermal-residual blob test, and every remaining parameter is a threshold tuned on validation. Its central observation is that when two people touch, their thermal signatures merge into ONE connected warm region while the person detector still reports TWO boxes; that disagreement is the contact signal, measured entirely in image space, so it sidesteps the homography noise (+/-15px, against a delta~18px decision threshold) that limits both homography-based contact detectors above. Full mathematical derivation -- adaptive residual threshold, the exactly-two-body merge predicate, the veto-based cross-camera quorum, and the 1-D morphological opening/closing on the decision stream -- is S1 §8. It is not a ThermalAlgorithm subclass and has no checkpoint, so it falls outside eval_all_detectors.py's registry-driven sweep; the table below is produced by a dedicated driver, scripts/eval_config_d.py, on the IDENTICAL 470 held-out frames every other contact detector in this report is scored on. That driver applies two protocol tightenings over the historical config-D runs, both of which make this number stricter rather than more flattering: (1) the raw-input MobileNet-SSD it depends on is retrained with the contact TEST scenes excluded -- the original runs trained the person detector on a within-scene timeline split that included frames from the very scenes contact was then scored on, which is harmless for a person detector but leaks into a contact evaluation; and (2) the morphology parameters (l_open, l_close) are tuned on non-test scenes only and applied unchanged to test. For reference, the historical figures were F1 60.8% on a single within-scene split and 44.3% pooled

over a 4-fold timeline cross-validation (per-fold 32/51/65/0%); the measured 52.7% below sits between them, as expected. **Config-D OUTPERFORMS the trained network on these frames: 52.7% F1 vs ThermoX3DDetector's 39.8% (S4), driven by far higher recall (83.6% vs 36.1%).** The two sit at opposite ends of the same tradeoff -- config-D catches most contacts but false-alarms on 46.8% of negative frames, ThermoX3D false-alarms on only 15.8% but misses two thirds of contacts -- so neither is simply 'better', and an operational choice between them is a decision about which failure is more tolerable in a ward. That a hand-written rule using NO contact training data beats a tuned network is itself the finding, and it reproduces the robust negative result from the original investigation: every learned variant of this rule underperformed the rule at this data scale (see reports/historical_investigations.md §4). **One asymmetry to disclose:** config-D's morphology was tuned on 3pp_surprise alone, because 2men_clash -- the freshly-annotated scene ThermoX3D validates on -- exists only as a per-scene contact_labels.csv and was never appended to the consolidated waveshare_work/contact_labels.csv that this driver reads. The held-out TEST frames are identical for both detectors, so the comparison above is sound, but the two tuning sets differ. Promoting config-D to a first-class detector under thermal_algorithms/contact_detection/ remains open work.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	61.1%	38.5%	83.6%	52.7%	N/A	N/A	N/A	CPU*
onnx_fp32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
fp16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
bf16	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--
int8	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--

Table 8. RBTCT (Rule-Based Temporal Contact Tracker) -- metrics by variant

onnx_fp32: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

fp16: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

bf16: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

int8: not available -- rule-based detector (is_trainable=False): no torch module and no ONNX graph exist, so there is nothing to cast or quantize -- its parameters are thresholds, not weights.

3. ONNX Export Scope

Only the learned sub-component of each detector is traced into ONNX. Classical-CV pre/post-processing is deterministic NumPy/OpenCV code, not a neural-net or sklearn estimator, and isn't part of any framework's ONNX graph by construction -- see each row's scope note.

FireSVMDetector

StandardScaler + SVC(probability=True) only -- Otsu segmentation and blob feature extraction are classical CV code, not exported.

ONNX path: C:\Users\guych\Desktop\Final Project\Weights\onnx\fire_svm_detector.onnx

HOGSVMDetector

LinearSVC decision function only -- the HOG descriptor and multi-scale sliding-window pyramid are classical CV code, not exported.

ONNX path: C:\Users\guych\Desktop\Final Project\Weights\onnx\hog_svm_detector.onnx

MobileNetSSDDetector

Full backbone + SSD heads. Anchor decoding + NMS stay in Python (thermal_algorithms.human_detection.mobilenet_ssd_anchors) -- apply identically to torch and ONNX raw outputs for a fair comparison.

ONNX path: C:\Users\guych\Desktop\Final Project\Weights\onnx\mobilenet_ssd_detector.onnx

MVSTGCNDetector

Graph-conv body only, at a FIXED representative actor count (max_actors=5, mask-padded exactly as real inference does). Detection + Kalman tracking + homography projection + graph construction (thermal_algorithms.contact_detection.mv_stgcn / multi_view/*) stay in Python -- not neural-net ops.

ONNX path: C:\Users\guych\Desktop\Final Project\Weights\onnx\mv_stgcn_detector.onnx

ThermoX3DDetector

Full 3D-CNN body. Per-camera frame buffering + global normalization stay in Python.

ONNX path: C:\Users\guych\Desktop\Final Project\Weights\onnx\thermo_x3d_detector.onnx

4. Unbiased (Balanced-Training) Results

The results above train/calibrate every detector on the corpus's NATURAL class ratio (fire ~28% positive, contact ~2.4%, human ~90% positive here in waveshare_work -- see S1). This section retrains/recalibrates 6 of those detectors (MVSTGNCNDetector excluded, per the project owner) on a genuinely resampled 50/50 positive/negative training set -- real sample-level resampling (thermal_algorithms/training/balance.py), not just class_weight loss-reweighting -- to separate "was this detector's behaviour a training-data-ratio artifact" from "is this a real limitation". Test scenes are UNCHANGED from the natural-ratio sections above in every case, so numbers here are directly comparable to S2's. This section reports the fp32 baseline only -- the ONNX/fp16/bf16/int8 sweep was run against the natural-ratio checkpoints in S2, so the absence of those rows here means 'not run for the balanced checkpoints', not 'not supported'; S2 carries the per-detector convertibility notes.

Human Detection

AdaptiveThresholdDetector

Calibrated for the FIRST time in this project (previously is_trainable=False, fit() a documented no-op, never validated against labeled data at all -- see S2's HOGSVMDetector/MobileNetSSDDetector caveats for the related empty_room-negatives-test-set fix this shares). Chose c_offset=1.528, min_area_pixels=4. Held-out (human_test_scenes + empty_room, matching HOGSVM/MobileNetSSD's own test-set convention): 86.2% acc / 86.6% prec / 97.3% rec / 91.6% F1, mean IoU 14.0%. False-alarm rate on empty_room: 203/384 = 52.9% -- meaningfully better than HOGSVMDetector's 98.7% but worse than MobileNetSSDDetector's false-alarm rate (see below). No natural-ratio counterpart exists for this detector (it was calibrated directly against balanced data in this pass, not re-calibrated from an existing baseline), so this is a first-time result, not a before/after comparison like the other five.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	86.2%	86.6%	97.3%	91.6%	14.0%	0.972	0.136	CPU*

Table 1. AdaptiveThresholdDetector -- balanced-training metrics

HOGSVMDetector

Retrained on a genuinely 50/50-resampled pool of 908 frames (454 person-positive + 454 negative -- capped by waveshare_work's scarce negative-frame pool, with empty_room EXCLUDED from training since it's used as an extra negative test scene -- see thermal_algorithms/training/balance.py's build_balanced_human_pool exclude_scenes param). class_weight="balanced" was kept (not set to None) for this retrain, unlike FireSVMDetector above: HOGSVMDetector.fit() does not train one sample per frame -- it emits one positive HOG patch per ground-truth bbox plus up to 5 random background patches PER FRAME regardless of that frame's own label, so the frame-level 50/50 resampling does not by itself produce a 50/50 patch-level ratio (closer to ~1:5 in practice); dropping the loss-reweighting on top would have silently reintroduced an uncorrected imbalance. **Held-out (human_test_scenes + empty_room): 77.8% acc / 77.8% prec / 100% rec / 87.5% F1, mean IoU 55.6% -- false-alarm rate on empty_room is 384/384 = 100% (tn=0), marginally WORSE than the natural-ratio run's 379/384 = 98.7%.** Balanced training did not help this detector at all -- consistent with its false-positive behaviour being driven by the dense, exhaustive sliding-window search itself (some candidate box on nearly every frame regardless of content) rather than the SVM decision boundary's class balance; rebalancing the training data cannot fix a structural property of the search procedure.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	77.8%	77.8%	100.0%	87.5%	55.6%	1505.766	1609.944	CPU*

Table 2. HOGSVMDetector -- balanced-training metrics

MobileNetSSDDetector

Retrained on the SAME balanced 908-frame human pool as HOGSVMdetector above (empty_room excluded from training). Held-out (human_test_scenes + empty_room): 98.0% acc / 98.4% prec / 99.1% rec / 98.7% F1, mean IoU 72.1% -- vs. the natural-ratio run's 97.2% / 96.6% / 100% / 98.2% (76.0%). ****A genuine improvement****: false-alarm rate on empty_room dropped from 48/384 (12.5%) to 22/384 (5.7%) -- more than half -- for a negligible recall cost (12 more missed frames out of 1344 positives) and a small mean-IoU decrease. Balanced training measurably helped this detector's false-positive suppression, the clearest net win in this whole balanced-training pass.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	98.0%	98.4%	99.1%	98.7%	72.1%	3.561	4.697	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*

Table 3. MobileNetSSDDetector -- balanced-training metrics

Fire Detection

OtsuFireDetector

Calibrated (grid search, see S2's OtsuFireDetector caveat for the methodology) against a genuinely 50/50-resampled pool (thermal_algorithms.training.balance.build_balanced_fire_pool) instead of the natural ~28% fire-positive corpus ratio -- chosen t_ign=39.2C (was 41.5C natural). Held-out on the SAME natural-ratio fire_test_scenes: 75.8% acc / 59.2% prec / 58.1% rec / 58.7% F1 (mean IoU 9.2%) vs. the natural-ratio calibration's 79.7% / 83.0% / 39.2% / 53.2% (7.0%). Balancing shifted the precision/recall tradeoff -- +18.9pp recall for -23.8pp precision -- with a net +5.5pp F1 gain: a real, disclosed tradeoff (catches more real fires, at the cost of more false alarms), not an unambiguous win.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	75.8%	59.2%	58.1%	58.7%	9.2%	1.769	0.189	CPU*

Table 4. OtsuFireDetector -- balanced-training metrics

FireSVMdetector

Retrained on a genuinely 50/50-resampled pool (28,896 examples: 14,448 fire-positive + 14,448 negative) instead of the natural ~28% ratio. Held-out: 70.3% acc / 48.8% prec / 15.0% rec / 23.0% F1 -- virtually UNCHANGED from the natural-ratio run's 71.6% / 57.5% / 15.0% / 23.8% (recall identical to 3 significant figures). Sample-level rebalancing did not move this detector's performance, most plausibly because the natural-ratio run already used sklearn's class_weight="balanced" (loss-reweighting) -- the balanced run keeps that setting too, since it's a genuine no-op on an already-even input pool (see thermal_algorithms/training/balance.py's build_balanced_fire_pool docstring), so this comparison isolates sample-level resampling from loss-reweighting cleanly: for this detector, loss-reweighting alone was already doing the correcting work that data-level rebalancing would otherwise do.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	70.3%	48.8%	15.0%	23.0%	N/A	3.696	3.361	CPU*

Table 5. FireSVMdetector -- balanced-training metrics

Contact Detection

ThermoX3DDetector

READ THIS BEFORE ANY CONTACT NUMBER IN THIS REPORT. Every contact figure quoted elsewhere comes from ONE fixed train/test split. The corpus contains 246 contact-positive frames spread over exactly FIVE scenes, and the fixed split locks three of those five into test permanently -- so single-split contact numbers are extremely fragile. scripts/cv_contact_detectors.py therefore re-scores every contact detector under leave-one-scene-out cross-validation (each positive scene is the test fold once; all other scenes train/tune), pooling the folds' frames. Results:

```
detector pooled_F1 recall TNR bal.acc MCC
always-alarm 0.432 1.000 0.000 0.500 0.000 <- trivial baseline
RBTCT 0.431 0.512 0.671 0.592 0.169
ThermoX3D 0.448 0.907 0.187 0.547 0.113
OR-ensemble 0.442 0.935 0.130 0.532 0.092
```

Finding 1 -- F1 is uninformative on this task. The pooled positive base rate is 27.5%, so a detector that simply alarms on every frame scores F1 0.432. All three real detectors score 0.431-0.448. Judged on F1 alone, none of them is distinguishable from a broken always-on detector, and any F1-based ranking between them is noise. F1 ignores true negatives, which is precisely the quantity that matters when the operational failure mode is alarm fatigue. **Finding 2 -- once true negatives are credited, the ranking inverts.** By balanced accuracy and MCC (both of which reward correctly rejecting negatives), RBTCT is the strongest detector (0.592 / 0.169), ThermoX3D is marginal (0.547 / 0.113), and the OR-ensemble is the WEAKEST (0.532 / 0.092) -- it inherits both detectors' false alarms while adding little recall. The 'run both, alarm if either fires' posture recommended by the earlier investigation is hereby measured and NOT supported. **Finding 3 -- every detector has a total-failure fold.** Per-fold F1: RBTCT [0.00, 0.69, 0.27, 0.48, 0.39], ThermoX3D [0.29, 0.61, 0.00, 0.95, 0.38]. RBTCT scores zero on 2men_clash; ThermoX3D scores zero on 2ppl_hug. A single split showing either detector at ~0.5 F1 conceals this entirely. Note also that ThermoX3D's 0.95 on 3pp_surprise is NOT skill: that scene is 95.7% positive, where always-alarm scores 0.978 -- the detector scored BELOW the trivial baseline on its best-looking fold. This CV reproduces the independent 4-fold result from the original 2026-07 investigation (pooled 44.3%, per-fold 32/51/65/0%) including the zero fold, from a separate implementation five weeks apart. Full data: reports/contact_cv_results.json.

REDONE 2026-08-09 -- the previous balanced checkpoint's 'identical to natural-ratio' result was a broken-training-loop artifact (both were degenerate constant classifiers; see S2's ThermoX3DDetector caveat for the full root-cause writeup), not a real finding about class imbalance. Fixed and retrained from scratch: persistent AdamW optimizer (was rebuilt fresh every chunk, discarding momentum state), checkpoint-level frozen normalisation stats (was recomputed per chunk from whatever data that chunk happened to hold), class-mixed super-chunks with a ratio cap (thermal_algorithms/training/balance.py's build_balanced_contact_pools/split_contact_pools_train_val/interleave_chunks -- replacing the single iter_balanced_contact_runs generator so train/val splitting happens at the parent-run level, before sub-chunking, avoiding near-duplicate frames landing on both sides), lower learning rate (1e-4, was 1e-3), a real validation split with early stopping (best-val-loss state restored), and an architecture change from T=16 to T=5 frames (input tensor (3,5,62,80)) per the project owner's direction. The training-loop bug is CONFIRMED fixed: recall went from 0% to ~100% and the model's raw confidence output now varies meaningfully with input (the dead-network signature -- bit-identical output regardless of input -- is gone). **CURRENT CHECKPOINT -- real-only + augmentation + Optuna (2026-08-09):** synthetic data was dropped entirely (see the failed experiment below) and the detector retrained on waveshare_work

alone, with augmentation standing in for corpus size: horizontal flip, +/-15 deg rotation, and additive $N(0, \sigma)$ noise, applied to RAW frames before preprocessing so σ stays interpretable in degrees C (thermal_algorithms/training/augment.py). Geometric transforms are drawn once per contiguous run -- constant across all 5 frames and all 3 cameras -- because a per-frame angle would inject artificial motion that this detector's temporal convolutions would learn as signal; noise is per-frame i.i.d., which is what sensor noise actually is. Two further directives: a window is labelled positive when ≥ 2 of its 5 triplets are contact-positive (replacing the old label-by-last-frame rule, under which one borderline frame flipped a whole window), and weights initialise as identity matrices (Dirac kernels for convs, partial identity for linear layers). 30 Optuna trials (TPE + median pruning) tuned the rest. **Held-out result: 71.7% acc / 44.4% prec / 36.1% rec / 39.8% F1 (tp=44, tn=293, fp=55, fn=78).** F1 is statistically flat against both earlier regimes (42.1%, 41.1%) but the OPERATING POINT moved decisively: precision nearly doubled (26.6% -> 44.4%) and the false-alarm rate collapsed from 96.6% to **15.8%** (fp 336 -> 55). This is the first contact checkpoint in this project that is not effectively an always-alarm detector -- a distinction F1 alone hides, and the one that matters for a psychiatric-ward alarm, where a 96% false-alarm rate is unusable regardless of recall. The cost is real and must not be glossed: recall fell to 36.1%, i.e. it now misses roughly two thirds of contact events, which is its own safety problem. Empirical notes from the search: identity init BEAT standard init (the directive was validated, not merely followed), and the search moved noise σ away from the specified 2.0 down to ~ 1.08 degC -- consistent with 2 degC approaching the corpus's own frame-wide std of 2.39 degC and beginning to bury the signal rather than regularise it. **The binding constraint is unchanged and bounds all of the above:** only two non-test scenes contain ANY contact positives -- 3pp_surprise (66, and flagged block-labelled/suspect by the label audit in reports/historical_investigations.md S5.1) and 2men_clash (58, freshly annotated). A leakage-free scene-level split puts one on each side, so every positive training window here derives from a single 66-frame clip. Augmentation multiplies frames; it cannot manufacture a second contact scenario, and the val-to-test recall drop (100% -> 36.1%) is exactly that generalisation gap. Full study in reports/contact_optuna_study.json; driver scripts/train_contact_real_optuna.py.

SUPERSEDED EXPERIMENT -- source-separated synth/real (2026-08-09): at the project owner's direction the data regime was then changed so that TRAINING draws only on synthetic data (synth_room_1..5), VALIDATION mixes held-out synthetic with all 15 real non-test contact scenes (including the freshly-annotated 2men_clash), and TEST remains real-only -- the goal being to stop spending scarce real contact data on training and spend it on the decision-threshold calibration instead. Training chunks and the validation set are both balanced to an EXACT 50/50 positive/negative WINDOW split. **Held-out result: 30.4% acc / 26.3% prec / 93.4% rec / 41.1% F1 (tp=114, tn=29, fp=319, fn=8) -- statistically indistinguishable from the previous mixed-source regime's 42.1% F1 / 26.6% precision. Moving real data out of training did NOT fix the precision problem.** The diagnostic that explains why is a measured DOMAIN GAP between the two sources: raw synthetic frames span only ~ 1 degree C (ambient ~ 29.9 C, std ~ 0.11 , people peaking ~ 31 C) whereas real frames span ~ 23 degrees C (13-36C, std ~ 2.39 , bodies at 36C against a structured background). After preprocessing, real data sits $\sim +7.9$ σ outside the synthetic training distribution. Per-source normalisation (each source scaled by its own statistics into a common $\sim N(0,1)$ space; the checkpoint saved under REAL statistics since all test/deployment input is real) recovers most of the numeric mismatch -- it cut validation loss from 7.09 to 0.81 -- but it cannot manufacture the background thermal STRUCTURE that synthetic scenes lack, and background structure is precisely what drives real-world false positives. The honest conclusion is that this synthetic corpus does not substitute for real contact data on this task; the binding constraint remains the ~ 250 real positive contact frames in the whole corpus (see reports/historical_investigations.md §3.2 and §4.1, where two independent earlier investigations reached the same data-limited conclusion). Reported as a feasibility result, not a benchmark.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
fp32_baseline	71.7%	44.4%	36.1%	39.8%	N/A	8.742	16.604	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)*
onnx_fp32	71.7%	44.4%	36.1%	39.8%	N/A	7.278	10.901	ONNX Runtime / CUDAExecutionProvider (NVIDIA GeForce RTX 5060 Laptop GPU)
fp16	71.7%	44.4%	36.1%	39.8%	N/A	9.775	16.908	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms	Measured on
bf16	72.1%	45.6%	38.5%	41.8%	N/A	9.978	14.960	NVIDIA GeForce RTX 5060 Laptop GPU (CUDA)
int8	N/A	N/A	N/A	N/A	N/A	N/A	N/A	--

Table 6. *ThermoX3DDetector -- balanced-training metrics*

int8: not available -- quantization of the working (balanced) checkpoint SUCCEEDS but the resulting graph fails to load: onnxruntime raises 'Node (/streams.2/block1/attention/channel/mlp/mlp.0_1/Gemm_MatMul_quant) Op (MatMulInteger) [ShapeInferenceError] Incompatible dimensions for matrix multiplication' -- the CBAM channel-attention MLP is the same sub-graph the quantizer already needed a DefaultTensorType work-around for. Note this is a DIFFERENT failure from the S2 checkpoint's, which aborts earlier during quantization itself.